

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Đồ Án Chuyên Ngành

HỆ THỐNG LMS TỰ ĐỘNG TẠO
MICRO-CONTENT & QUIZ
BẰNG GENERATIVE AI

(Tuần 1 – 15)

GV hướng dẫn: Tên Giảng Viên

SV thực hiện: Phùng Xương Cận - 2210348



Table 1: Bảng phân công nhiệm vụ và nhận xét của giảng viên.

STT	Họ tên	MSSV	Nhiệm vụ	Đóng góp	Ký tên
3	Phùng Xương Cận	2210348	Frontend Giảng viên & Học viên; Demo & Kiểm thử	100%	
Nhận xét của giảng viên:				Ký tên	
(Đây là khu vực nhận xét của giảng viên)					

LỜI CAM ĐOAN

Chúng tôi xin cam đoan rằng đây là công trình nghiên cứu của nhóm chúng tôi trong thời gian qua. Các số liệu và kết quả trình bày trong báo cáo là trung thực, xuất phát từ tình hình thực tế. Chúng tôi xin hoàn toàn chịu trách nhiệm về lời cam đoan này.

Thành phố Hồ Chí Minh, tháng ... năm 2026

Nguyễn Nam Phương

Nguyễn Minh Triết

Phùng Xương Cậ

LỜI CẢM ƠN

Trước tiên, chúng tôi xin gửi lời cảm ơn chân thành đến **Thầy/Cô [Tên Giảng Viên Hướng Dẫn]**, người đã tận tình hướng dẫn, định hướng và hỗ trợ nhóm trong suốt quá trình thực hiện đề tài. Những góp ý kịp thời và sâu sắc của Thầy/Cô đã giúp chúng tôi vượt qua nhiều thách thức kỹ thuật và hoàn thiện sản phẩm theo đúng định hướng.

Chúng tôi cũng xin cảm ơn **Khoa Khoa học và Kỹ thuật Máy tính**, Trường Đại học Bách Khoa TP.HCM đã tạo điều kiện về cơ sở vật chất và môi trường học thuật để nhóm có thể tiến hành nghiên cứu và hiện thực đề tài.

Xin cảm ơn **các bạn sinh viên** đã tham gia khảo sát và thử nghiệm sản phẩm, cũng như đóng góp những phản hồi thực tế và chân thực.

Cuối cùng, chúng tôi xin cảm ơn **gia đình và bạn bè** đã luôn ủng hộ và động viên chúng tôi trong suốt thời gian thực hiện đề tài.

Do giới hạn về thời gian và kinh nghiệm, báo cáo không tránh khỏi những thiếu sót. Chúng tôi rất mong nhận được sự góp ý từ Thầy/Cô và Hội đồng để hoàn thiện hơn trong Giai đoạn 2.

Thành phố Hồ Chí Minh, tháng ... năm 2026

Nhóm thực hiện

TÓM TẮT ĐỀ TÀI

Tên đề tài: Hệ thống LMS tự động tạo Micro-Content & Quiz bằng Generative AI

Giai đoạn: Giai đoạn 1 (Tuần 1–15)

Giảng viên hướng dẫn: [Tên Giảng Viên]

Nhóm thực hiện: Nguyễn Nam Phương (2212698), Nguyễn Minh Triết (2213608),
Phùng Xương Cạn (2210348)

Bối cảnh và vấn đề

Trong bối cảnh giáo dục hiện đại, việc chuyển đổi tài liệu học thuật (giáo trình, slide bài giảng, tài liệu PDF) thành nội dung Micro-learning vẫn đang được thực hiện thủ công bởi giảng viên — tốn từ 2 đến 4 giờ cho mỗi chương và không đảm bảo tính nhất quán. Sự ra đời của các Mô hình Ngôn ngữ Lớn (LLM) như GPT-4o và Gemini mở ra cơ hội tự động hóa quy trình này.

Giải pháp đề xuất

Nhóm xây dựng một hệ thống Micro-learning tích hợp AI với pipeline tự động:

- Document Parsing:** Trích xuất text và cấu trúc từ PDF, DOCX, PPTX bằng các thư viện chuyên dụng.
- Text Chunking:** Phân đoạn văn bản theo cấu trúc heading ngữ nghĩa (Heading-based Chunking); áp dụng sliding window với overlap cho section dài.
- Vector Embedding:** Tạo embedding 768 chiều bằng mô hình BAAI/bge-m3 đa ngôn ngữ; lưu vào Qdrant Vector Database để hỗ trợ tìm kiếm ngữ nghĩa.
- Sinh nội dung AI:** Gọi LLM API (GPT-4o-mini) với prompt được thiết kế để sinh bài tóm tắt (Summary) và bộ câu hỏi trắc nghiệm (Quiz) cho từng chunk.
- Human-in-the-Loop:** Giao diện web cho giảng viên review, chỉnh sửa và publish; giao diện học viên xem Card và làm Quiz.

Hệ thống sử dụng kiến trúc **Domain-Driven Design (DDD)** với giao thức **gRPC** cho phép dễ dàng thay thế LLM provider hoặc thêm parser mới mà không ảnh hưởng đến logic nghiệp vụ.

Kết quả đạt được

- Pipeline xử lý tài liệu 25 trang PDF trong khoảng ... giây, sinh ra ... Card tóm tắt và ... câu hỏi Quiz.
- Tỷ lệ nội dung Summary không cần chỉnh sửa: $\sim \dots \%$.
- Tỷ lệ câu Quiz không cần chỉnh sửa: $\sim \dots \%$.
- Demo thành công kịch bản end-to-end: upload một chương giáo trình Machine Learning $\rightarrow$ nhận khóa học Micro-learning hoàn chỉnh.

Từ khóa: Micro-learning, Large Language Model, Text Chunking, Prompt Engineering, Vector Embedding, RAG, EdTech.

Mục lục

Danh mục bảng biểu	11
Danh mục hình ảnh	13
Danh mục từ viết tắt	14
1 Tổng quan	16
1.1 Mô tả đề tài	16
1.1.1 Khảo sát sơ bộ	16
1.1.2 Mục tiêu đề tài	16
1.1.3 Định hướng sản phẩm	17
1.1.4 Tiêu chí sản phẩm	17
1.1.5 Mô hình dự kiến	17
1.2 Phân công nhiệm vụ	18
1.2.1 Phân công Giai đoạn 1	18
1.3 Cấu trúc báo cáo	18
1.4 Tổng kết Chương 1	19
2 Khảo sát và phân tích đề tài	20
2.1 Khảo sát thực trạng	20
2.1.1 Khảo sát quy trình giảng viên	20
2.1.2 Khảo sát học viên	20
2.2 Khảo sát giải pháp tương tự	20
2.2.1 Coursera — AI-generated Summaries	20
2.2.2 Khan Academy — Khanmigo (AI Tutor)	20
2.2.3 Quizlet AI — Smart Notes	20
2.2.4 So sánh và đánh giá các giải pháp	21
2.3 User Stories	21
2.3.1 Giảng viên	21
2.3.2 Học viên	22
2.3.3 Quản trị viên	22
2.4 Yêu cầu nghiệp vụ	23
2.4.1 BR-01: Xác thực và phân quyền người dùng	23
2.4.2 BR-02: Quản lý tài liệu và nội dung học tập	23
2.4.3 BR-03: Xử lý và sinh nội dung AI	23
2.4.4 BR-04: Trải nghiệm học tập và đánh giá	23
2.5 Yêu cầu tính năng	23

2.5.1	FR-01: Đăng nhập / Đăng xuất / Đăng ký tài khoản	24
2.5.2	FR-02: Quản lý tài liệu	24
2.5.3	FR-03: Xử lý tài liệu tự động (AI Pipeline)	24
2.5.4	FR-04: Review và publish khóa học	24
2.5.5	FR-05: Trải nghiệm học tập	25
2.6	Yêu cầu phi tính năng	25
2.7	Tổng kết Chương 2	25
3	Cơ sở lý thuyết	26
3.1	Lý thuyết công nghệ sử dụng	26
3.1.1	Mô hình Ngôn ngữ Lớn (Large Language Model)	26
3.1.2	Prompt Engineering	26
3.1.3	Text Chunking	27
3.1.4	Vector Embedding	27
3.1.5	Domain-Driven Design (DDD)	27
3.2	Cân nhắc lựa chọn công nghệ	28
3.2.1	Lựa chọn LLM API: GPT-4o-mini vs Gemini Flash	28
3.2.2	Lựa chọn chiến lược Chunking	28
3.2.3	Lựa chọn Vector Database: Qdrant vs Weaviate vs Pinecone	29
3.2.4	Lựa chọn Embedding Model: bge-m3 vs text-embedding-3-small	29
3.3	Tổng kết Chương 3	29
4	Thiết kế hệ thống	30
4.1	Thiết kế kiến trúc hệ thống	30
4.1.1	Tổng quan kiến trúc	30
4.1.2	Kiến trúc phân tầng (Layered Architecture)	30
4.1.3	Chi tiết các service	31
4.1.4	Bài toán thiết kế và giải pháp	32
4.1.5	Thiết kế luồng dữ liệu (Data Flow)	35
4.2	Thiết kế giao tiếp giữa các service (API Design)	36
4.2.1	Tổng quan giao thức gRPC	36
4.2.2	Định nghĩa Service Protobuf	37
4.3	Thiết kế cơ sở dữ liệu	39
4.3.1	Lược đồ ERD tổng quan (hai giai đoạn)	39
4.3.2	Các bảng nghiệp vụ cốt lõi	39
4.3.3	Các bảng bổ sung cho Video và Chatbot	41
4.3.4	Mức độ chuẩn hóa và trade-off	42
4.3.5	Chiến lược index	43
4.4	Thiết kế Sequence Diagram	43
4.4.1	Sequence Diagram: Upload và xử lý tài liệu	43

4.4.2	Sequence Diagram: Học viên làm Quiz	44
4.4.3	Sequence Diagram: RAG Chatbot	44
4.4.4	Sequence Diagram: Video Processing	45
4.5	Thiết kế bảo mật	46
4.5.1	Xác thực và phân quyền	46
4.5.2	Bảo mật dữ liệu và API	46
4.6	Thiết kế triển khai (Deployment Architecture)	47
4.6.1	Docker Compose topology	47
4.6.2	Network và Volume	47
4.6.3	Định hướng mở rộng sang Kubernetes (sau GD2)	47
4.7	Thiết kế giao diện	48
4.7.1	Bộ nhận diện thương hiệu	48
4.7.2	Sơ đồ trang (Sitemap)	48
4.7.3	Wireframe các màn hình chính	49
4.8	Thiết kế Use Case Diagram	50
4.8.1	Nhóm use case Xác thực	50
4.8.2	Nhóm use case Quản lý tài liệu và nội dung	50
4.8.3	Nhóm use case Học tập	50
4.8.4	Nhóm use case Quản trị	51
4.9	Thiết kế Activity Diagram	51
4.9.1	Quy trình upload và xử lý tài liệu	51
4.9.2	Quy trình AI Pipeline (Job Queue + State Machine)	51
4.9.3	Quy trình review và publish	51
4.9.4	Quy trình học viên học bài và làm Quiz	52
4.9.5	Quy trình Chatbot RAG	52
4.10	Tổng kết Chương 4	52
5	Hiện thực hệ thống	54
5.1	Nền tảng sử dụng	54
5.1.1	Python 3.11 và gRPC	54
5.1.2	MinIO Object Storage	54
5.1.3	Qdrant Vector Database	54
5.1.4	Redis Cache và Job Queue	54
5.1.5	React + TypeScript — Frontend	54
5.1.6	Docker và Docker Compose	54
5.2	Hiện thực AI Pipeline	55
5.2.1	Module Document Parsing	55
5.2.2	Module Text Chunking	55
5.2.3	Module sinh nội dung AI	55



5.2.4	Use Case ProcessDocument	55
5.3	Hiện thực Web Application	56
5.3.1	Giao diện Giảng viên	56
5.3.2	Giao diện Học viên	56
5.4	Tổng kết Chương 5	56
6	Kiểm thử hệ thống	57
6.1	Kế hoạch kiểm thử	57
6.1.1	Mục tiêu	57
6.1.2	Phương pháp	57
6.1.3	Phạm vi	57
6.2	Kiểm thử tính năng (Functional Testing)	58
6.2.1	Cấu hình môi trường kiểm thử	58
6.2.2	Kiểm thử Hộp đen (Black-box Testing)	58
6.3	Kiểm thử phi tính năng	60
6.3.1	Cấu hình môi trường kiểm thử	60
6.3.2	Kiểm thử hiệu năng (Benchmark & Load Test)	60
6.3.3	Kiểm thử chất lượng nội dung AI	61
6.4	Tổng kết Chương 6	61
7	Kết luận	62
7.1	Kết quả đạt được	62
7.2	Điểm cần cải thiện	62
7.3	Định hướng phát triển	63
	Phụ lục	65

Danh sách bảng

1	Bảng phân công nhiệm vụ và nhận xét của giảng viên.	1
3	Tiêu chí chấp nhận sản phẩm Giai đoạn 1.	17
4	Phân công nhiệm vụ Giai đoạn 1.	18
5	So sánh các giải pháp tương tự với hệ thống đề xuất.	21
6	User Stories của Giảng viên.	21
7	User Stories của Học viên.	22
8	User Stories của Quản trị viên.	23
9	Yêu cầu phi tính năng (NFR).	25
10	Các kỹ thuật Prompt Engineering và ứng dụng trong đề tài.	26
11	So sánh GPT-4o-mini và Gemini Flash.	28
12	So sánh các chiến lược Chunking cho tài liệu học thuật.	28
13	So sánh các Vector Database.	29
14	So sánh mô hình Embedding.	29
15	Mô tả bốn tầng kiến trúc của hệ thống.	31
16	Năm bài toán thiết kế và giải pháp tương ứng.	32
17	Các trạng thái trong State Machine của AI Pipeline.	33
18	Danh sách Port và Adapter trong hệ thống.	34
19	Định nghĩa RPC cho Auth Service.	37
20	Định nghĩa RPC cho Document Service.	37
21	Định nghĩa RPC cho Content Service.	38
22	Định nghĩa RPC cho Learning Service.	38
23	Định nghĩa RPC cho Chat Service.	38
24	Mô tả bảng <code>users</code>	39
25	Mô tả bảng <code>documents</code>	39
26	Mô tả bảng <code>chunks</code>	40
27	Mô tả bảng <code>cards</code> và <code>quiz_questions</code>	40
28	Mô tả bảng <code>courses</code>	41
29	Mô tả bảng <code>media_clips</code>	41
30	Mô tả bảng <code>transcripts</code>	41
31	Mô tả bảng <code>chat_sessions</code> và <code>chat_messages</code>	42
32	Mô tả bảng <code>learning_progress</code>	42
33	Mô tả bảng <code>quiz_attempts</code>	42
34	Danh sách index quan trọng.	43
35	Ma trận phân quyền theo vai trò.	46
36	Danh sách Docker containers.	47
37	Bảng màu sắc và typography của hệ thống.	48
38	Các phương pháp kiểm thử được áp dụng.	57

39	Cấu hình môi trường kiểm thử tính năng.	58
40	Test cases cho FR-01: Xác thực người dùng.	58
41	Test cases cho FR-02/03: Upload và xử lý tài liệu.	59
42	Test cases cho FR-04/05: Review, publish và học tập.	59
43	Cấu hình môi trường kiểm thử phi tính năng.	60
44	Kết quả kiểm thử hiệu năng API (50 concurrent users, locust).	60
45	Kết quả kiểm thử thời gian xử lý pipeline (tài liệu thực tế).	61
46	Kết quả đánh giá chất lượng nội dung AI (N=... giảng viên).	61
47	Kết quả SUS và task completion.	61
48	Tổng kết kết quả Giai đoạn 1 theo tiêu chí đề ra.	62

Danh sách hình

1	Kết quả khảo sát nhu cầu của giảng viên và học viên.	16
2	Mô hình tổng quát của sản phẩm.	17
3	Kết quả khảo sát giảng viên về quy trình tạo nội dung kiểm tra.	20
4	Kết quả khảo sát học viên về thói quen học tập.	20
5	Kiến trúc tổng quan hệ thống Micro-learning.	30
6	Kiến trúc phân tầng của hệ thống.	30
7	State Machine của AI Pipeline.	33
8	Kiến trúc Hexagonal — Port/Adapter cho AI Pipeline.	33
9	Kiến trúc RAG Pipeline cho Chatbot Tutor.	34
10	Luồng dữ liệu Text Processing: từ Upload đến Card + Quiz.	35
11	Luồng dữ liệu: từ Video Upload đến Clip + Card + Quiz.	35
12	Luồng dữ liệu RAG Chatbot.	36
13	Lược đồ ERD tổng quan. Bảng in nghiêng chỉ áp dụng khi input là video/audio.	39
14	Sequence Diagram: Giảng viên upload tài liệu và pipeline xử lý.	43
15	Sequence Diagram: Học viên làm Quiz và nhận phản hồi.	44
16	Sequence Diagram: Học viên hỏi Chatbot RAG.	44
17	Sequence Diagram: Xử lý Video.	45
18	Luồng xác thực JWT.	46
19	Docker Compose topology — tất cả service chạy trong container.	47
20	Sơ đồ trang (Sitemap) của toàn bộ hệ thống (hai giai đoạn).	48
21	Wireframe Dashboard Giảng viên — hiển thị tổng quan tài liệu và trạng thái pipeline.	49
22	Wireframe Review — layout 2 cột: bên trái là text gốc, bên phải là nội dung AI sinh.	49
23	Wireframe Card view — một Card hiển thị tại một thời điểm, navigation bằng phím mũi tên.	49
24	Wireframe Chatbot — panel bên phải trong màn hình học.	50
25	Use Case Diagram tổng quan (hai giai đoạn).	50
26	Use Case Diagram — Xác thực người dùng.	50
27	Use Case Diagram — Quản lý tài liệu và nội dung AI.	50
28	Use Case Diagram — Học tập và kiểm tra.	50
29	Use Case Diagram — Quản trị hệ thống.	51
30	Activity Diagram: Upload và xử lý tài liệu.	51
31	Activity Diagram: AI Pipeline chi tiết (Text và Video).	51
32	Activity Diagram: Review và publish khóa học.	51
33	Activity Diagram: Học viên xem Card, xem Video (nếu có) và làm Quiz.	52



34	Activity Diagram: Học viên hỏi Chatbot RAG.	52
35	Màn hình Dashboard Giảng viên.	56
36	Màn hình Upload tài liệu.	56
37	Màn hình theo dõi tiến trình xử lý AI (4 bước).	56
38	Màn hình Review nội dung AI (layout 2 cột).	56
39	Màn hình danh sách khóa học của Học viên.	56
40	Màn hình xem bài học dạng Card.	56
41	Màn hình làm Quiz (hiển thị phản hồi tức thì sau khi chọn đáp án).	56

Danh mục từ viết tắt

Từ viết tắt	Nghĩa đầy đủ
AI	Artificial Intelligence — Trí tuệ nhân tạo
ANN	Approximate Nearest Neighbor — Tìm kiếm láng giềng gần đúng
API	Application Programming Interface — Giao diện lập trình ứng dụng
AQG	Automatic Question Generation — Sinh câu hỏi tự động
BR	Business Requirement — Yêu cầu nghiệp vụ
BPMN	Business Process Model and Notation — Ký hiệu mô hình hóa quy trình nghiệp vụ
DDD	Domain-Driven Design — Thiết kế hướng nghiệp vụ
DOCX	Document Format of Microsoft Word (.docx)
DTO	Data Transfer Object — Đối tượng truyền dữ liệu
EdTech	Educational Technology — Công nghệ giáo dục
FR	Functional Requirement — Yêu cầu tính năng
gRPC	Google Remote Procedure Call — Giao thức gọi thủ tục từ xa của Google
HITL	Human-in-the-Loop — Con người trong vòng lặp
JWT	JSON Web Token — Token xác thực dạng JSON
LLM	Large Language Model — Mô hình ngôn ngữ lớn
LMS	Learning Management System — Hệ thống quản lý học tập
MCQ	Multiple Choice Question — Câu hỏi trắc nghiệm
MinIO	Minimum Input/Output — Hệ thống lưu trữ đối tượng
NFR	Non-Functional Requirement — Yêu cầu phi tính năng
NLP	Natural Language Processing — Xử lý ngôn ngữ tự nhiên
PDF	Portable Document Format — Định dạng tài liệu di động
PPTX	PowerPoint Presentation Format (.pptx)
Protobuf	Protocol Buffers — Định dạng tuần tự hóa dữ liệu nhị phân của Google
RAG	Retrieval-Augmented Generation — Sinh nội dung tăng cường bằng truy xuất
RBAC	Role-Based Access Control — Kiểm soát truy cập theo vai trò



Từ viết tắt	Nghĩa đầy đủ
REST	Representational State Transfer — Kiến trúc truyền thông web
SUS	System Usability Scale — Thang đo khả năng sử dụng hệ thống
UC	Use Case — Kịch bản sử dụng
UI/UX	User Interface / User Experience — Giao diện / Trải nghiệm người dùng
VDB	Vector Database — Cơ sở dữ liệu véc-tơ

1 Tổng quan

1.1 Mô tả đề tài

1.1.1 Khảo sát sơ bộ

Để hiểu rõ bài toán trước khi đề xuất giải pháp, nhóm đã thực hiện khảo sát sơ bộ với hai nhóm đối tượng:

Phía Giảng viên: Qua trao đổi với ... giảng viên tại Khoa Khoa học và Kỹ thuật Máy tính, ĐHBK TP.HCM, nhóm ghi nhận:

- ...% giảng viên phải đầu tư hơn 2 giờ để chuyển đổi một chương tài liệu thành nội dung kiểm tra.
- ...% không có công cụ hỗ trợ, phải làm hoàn toàn thủ công (soạn Word, nhập Moodle).
- ...% bày tỏ sẵn sàng dùng công cụ AI nếu kết quả đạt $\geq 70\%$ chất lượng so với làm tay.

Phía Học viên: Qua khảo sát ... sinh viên đại học:

- ...% cho rằng tài liệu học quá dài, khó tập trung khi ôn tập.
- ...% muốn có bài kiểm tra nhanh sau mỗi section để tự đánh giá mức độ hiểu bài.
- ...% đã từng dùng Quizlet hoặc flashcard để học.

Figure 1: Kết quả khảo sát nhu cầu của giảng viên và học viên.

Kết quả khảo sát xác nhận rõ khoảng trống: **thiếu công cụ tự động hóa việc tạo nội dung Micro-learning từ tài liệu sẵn có.**

1.1.2 Mục tiêu đề tài

Mục tiêu tổng quát: Xây dựng hệ thống Micro-learning thông minh có khả năng tự động chuyển đổi tài liệu học thuật (PDF, DOCX, PPTX) thành nội dung học tập dạng thẻ (Card) kèm bộ câu hỏi trắc nghiệm (Quiz), với sự kiểm soát và phê duyệt của giảng viên (*Human-in-the-Loop*).

Mục tiêu cụ thể Giai đoạn 1 (Tuần 1–15):

1. Nghiên cứu và thiết kế hệ thống prompt hiệu quả cho tác vụ tóm tắt (Summary) và tạo câu hỏi trắc nghiệm (Quiz) từ văn bản.

2. Xây dựng module **Text Chunking** phân đoạn văn bản theo cấu trúc ngữ nghĩa của tài liệu.
3. Xây dựng **Backend AI Pipeline** hoàn chỉnh: Upload $\rightarrow$ Parse $\rightarrow$ Chunk $\rightarrow$ Embed $\rightarrow$ Generate.
4. Xây dựng **giao diện web** cơ bản cho Giảng viên và Học viên.
5. **Demo** end-to-end: upload một chương sách giáo khoa và nhận lại khóa học Micro-learning hoàn chỉnh trong thời gian thực.

1.1.3 Định hướng sản phẩm

Sản phẩm hướng đến **nền tảng web SaaS** (Software as a Service) phục vụ các tổ chức giáo dục đại học và trung học phổ thông tại Việt Nam. Trong giai đoạn đầu, tập trung vào thị trường trong trường đại học.

Mô hình sử dụng:

- Giảng viên đăng nhập, upload tài liệu, nhận kết quả AI và publish.
- Học viên đăng nhập, học theo Card và làm Quiz để kiểm tra kiến thức.
- Quản trị viên quản lý tài khoản và theo dõi thống kê hệ thống.

1.1.4 Tiêu chí sản phẩm

Table 3: Tiêu chí chấp nhận sản phẩm Giai đoạn 1.

#	Tiêu chí	Ngưỡng	Cách đo
1	Thời gian xử lý tài liệu 20 trang	≤ 3 phút	Đo thực tế với 5 tài liệu mẫu
2	Tỷ lệ Summary đạt chất lượng	$\geq 70\%$	Giảng viên đánh giá (Likert 1–5, ≥ 3 là đạt)
3	Tỷ lệ Quiz đạt chất lượng	$\geq 65\%$	Tương tự trên
4	API response time (xem Card, Quiz)	$\leq 500\text{ms}$	Đo với 50 concurrent users
5	SUS Score giao diện học viên	$\geq 68/100$	Khảo sát 10 sinh viên
6	Coverage unit test	$\geq 70\%$	<code>pytest -cov</code>

1.1.5 Mô hình dự kiến

Figure 2: Mô hình tổng quát của sản phẩm.

1.2 Phân công nhiệm vụ

1.2.1 Phân công Giai đoạn 1

Table 4: Phân công nhiệm vụ Giai đoạn 1.

#	Thành viên	MSSV	Nhiệm vụ
1	Nguyễn Nam Phương	2212698	Nghiên cứu Prompt Engineering; thiết kế và thử nghiệm các prompt template; tích hợp LLM API (OpenAI/Gemini); viết chương 1 và 3.
2	Nguyễn Minh Triết	2213608	Xây dựng module Text Chunking và Document Parsing (PDF, DOCX, PPTX); thiết kế kiến trúc DDD; viết chương 4 và 5.
3	Phùng Xương Cận	2210348	Xây dựng giao diện web (Frontend Giảng viên & Học viên); triển khai Docker Compose; thực hiện kiểm thử; viết chương 2 và 6.

1.3 Cấu trúc báo cáo

Báo cáo được tổ chức thành 7 chương:

- **Chương 1 — Tổng quan:** Khảo sát sơ bộ, mục tiêu, tiêu chí sản phẩm và phân công nhiệm vụ.
- **Chương 2 — Khảo sát và phân tích:** Khảo sát thực trạng chi tiết, phân tích giải pháp tương tự, user stories và đặc tả yêu cầu (BR / FR / NFR).
- **Chương 3 — Cơ sở lý thuyết:** Lý thuyết LLM, Prompt Engineering, Text Chunking, Vector Embedding và lý giải lựa chọn công nghệ.
- **Chương 4 — Thiết kế hệ thống:** Kiến trúc, cơ sở dữ liệu, giao diện (Sitemap), use case diagram và activity diagram.
- **Chương 5 — Hiện thực hệ thống:** Nền tảng sử dụng, mã nguồn các module chính, giao diện web.
- **Chương 6 — Kiểm thử hệ thống:** Kiểm thử tính năng (black-box), kiểm thử phi tính năng (load test, chất lượng AI).
- **Chương 7 — Kết luận:** Kết quả, điểm cần cải thiện và định hướng Giai đoạn 2.



1.4 Tổng kết Chương 1

Chương 1 đã trình bày tổng quan về đề tài: bối cảnh và động cơ từ kết quả khảo sát sơ bộ, mục tiêu cụ thể của Giai đoạn 1, tiêu chí chấp nhận sản phẩm và phân công nhiệm vụ giữa các thành viên. Chương tiếp theo sẽ đi sâu vào khảo sát thực trạng và phân tích yêu cầu chi tiết của hệ thống.

2 Khảo sát và phân tích đề tài

2.1 Khảo sát thực trạng

2.1.1 Khảo sát quy trình giảng viên

Nhóm tiến hành phỏng vấn và khảo sát online với ... giảng viên đại học, tập trung vào ba câu hỏi chính: thời gian soạn nội dung kiểm tra, công cụ đang sử dụng, và mức độ sẵn sàng dùng AI.

Figure 3: Kết quả khảo sát giảng viên về quy trình tạo nội dung kiểm tra.

Nhận xét: Quy trình thủ công hiện tại tiêu tốn trung bình ... giờ/chương và hoàn toàn không có sự hỗ trợ của công cụ AI. Đây là pain point rõ ràng mà hệ thống đề xuất cần giải quyết.

2.1.2 Khảo sát học viên

Figure 4: Kết quả khảo sát học viên về thói quen học tập.

Nhận xét: Phần lớn học viên mong muốn nội dung ngắn gọn và kiểm tra tức thì — phù hợp hoàn toàn với triết lý Micro-learning của hệ thống đề xuất.

2.2 Khảo sát giải pháp tương tự

2.2.1 Coursera — AI-generated Summaries

Coursera gần đây bổ sung tính năng AI tóm tắt transcript bài giảng video, hiển thị bên cạnh video. Tuy nhiên, tính năng này chỉ áp dụng cho video có sẵn trên nền tảng và không cho phép giảng viên bên ngoài tải tài liệu lên để xử lý.

2.2.2 Khan Academy — Khanmigo (AI Tutor)

Khanmigo là chatbot AI tích hợp vào Khan Academy, hỗ trợ học viên giải thích bài tập theo phong cách Socratic (đặt câu hỏi dẫn dắt thay vì đưa đáp án thẳng). Giải pháp này không tự động sinh Quiz từ tài liệu mới và chỉ hỗ trợ tiếng Anh.

2.2.3 Quizlet AI — Smart Notes

Quizlet cho phép người dùng nhập hoặc paste văn bản, sau đó AI tự động tạo flashcard và câu hỏi trắc nghiệm. Ưu điểm là đơn giản, nhanh. Nhược điểm: không xử lý được file

PDF/DOCX phức tạp, không tích hợp vào quy trình giảng dạy (không có flow: upload → review → publish → học viên học).

2.2.4 So sánh và đánh giá các giải pháp

Table 5: So sánh các giải pháp tương tự với hệ thống đề xuất.

Tiêu chí	Coursera AI	Khanmigo	Quizlet AI	Notion AI	Hệ thống đề xuất
Upload tài liệu riêng	Không	Không	Có (text)	Có	Có (PDF/DOCX/PPTX)
Sinh Quiz tự động	Không	Không	Có	Không	Có
Review trước publish	—	—	Không	Không	Có
Hỗ trợ tiếng Việt	Không	Không	Một phần	Có	Có
Giao diện học viên	Có	Có	Có	Không	Có
Chi phí	Cao	Cao	Miễn phí + Pro	Có phí	Tự host

Kết luận: Không có giải pháp nào hiện tại đáp ứng đầy đủ ba tiêu chí: (1) upload tài liệu riêng, (2) có quy trình review Human-in-the-Loop, và (3) hỗ trợ tiếng Việt. Đây là khoảng trống mà đề tài hướng đến lấp đầy.

2.3 User Stories

2.3.1 Giảng viên

Table 6: User Stories của Giảng viên.

#	User Story	Tiêu chí chấp nhận
1	Là giảng viên, tôi muốn upload file PDF/DOCX để AI tự động tạo nội dung Micro-learning, giúp tôi tiết kiệm thời gian soạn bài.	Hệ thống nhận file $\leq 50\text{MB}$; thông báo lỗi rõ nếu sai định dạng; pipeline khởi động ≤ 5 giây sau upload.
2	Là giảng viên, tôi muốn theo dõi tiến trình xử lý AI để biết khi nào có thể review.	Hiển thị 4 bước xử lý; cập nhật tự động; không cần refresh.

#	User Story	Tiêu chí chấp nhận
3	Là giảng viên, tôi muốn chỉnh sửa nội dung AI trước khi publish để đảm bảo chính xác học thuật.	Chỉnh sửa inline trực tiếp; lưu tức thì; có lịch sử sửa đổi.
4	Là giảng viên, tôi muốn publish khóa học chỉ khi đã review xong để học viên không thấy nội dung chưa được kiểm tra.	Nút Publish chỉ active sau khi ≥ 1 section được review.
5	Là giảng viên, tôi muốn xem thống kê học viên (ai đã học, điểm Quiz) để đánh giá hiệu quả nội dung.	Dashboard hiển thị: số học viên, tỷ lệ hoàn thành, điểm Quiz TB.

2.3.2 Học viên

Table 7: User Stories của Học viên.

#	User Story	Tiêu chí chấp nhận
1	Là học viên, tôi muốn xem danh sách khóa học và tiến độ cá nhân để lên kế hoạch học tập.	Progress bar cá nhân trên mỗi card khóa học; lọc theo trạng thái.
2	Là học viên, tôi muốn học nội dung dạng Card ngắn gọn để có thể học mọi lúc, mọi nơi.	Card ≤ 200 từ; swipe gesture (mobile); phím mũi tên (desktop).
3	Là học viên, tôi muốn làm Quiz và nhận phản hồi tức thì để biết mình đã hiểu đúng chưa.	Hiển thị đúng/sai + giải thích ngay sau khi chọn; không thể đổi đáp án.
4	Là học viên, tôi muốn tìm kiếm nội dung bằng câu hỏi tự nhiên để ôn tập đúng phần cần thiết.	Nhập query; hệ thống trả về top-5 Card liên quan trong ≤ 2 giây.

2.3.3 Quản trị viên

Table 8: User Stories của Quản trị viên.

#	User Story	Tiêu chí chấp nhận
1	Là quản trị viên, tôi muốn quản lý tài khoản người dùng (tạo, khóa, đổi vai trò) để vận hành hệ thống.	CRUD tài khoản; đổi vai trò INSTRUCTOR/LEARNER; khóa tài khoản vi phạm.
2	Là quản trị viên, tôi muốn xem log chi phí API để kiểm soát ngân sách sử dụng LLM.	Dashboard chi phí theo ngày/tháng; cảnh báo khi vượt ngưỡng.

2.4 Yêu cầu nghiệp vụ

Yêu cầu nghiệp vụ (Business Requirements — BR) mô tả *what* hệ thống phải làm từ góc nhìn nghiệp vụ, không đề cập chi tiết kỹ thuật.

2.4.1 BR-01: Xác thực và phân quyền người dùng

Hệ thống phải phân biệt rõ ràng ba vai trò: Giảng viên, Học viên và Quản trị viên. Mỗi vai trò chỉ được thực hiện các thao tác thuộc phạm vi của mình. Người dùng phải xác thực trước khi truy cập bất kỳ chức năng nào.

2.4.2 BR-02: Quản lý tài liệu và nội dung học tập

Giảng viên phải có khả năng đưa tài liệu vào hệ thống và nhận lại nội dung Micro-learning. Nội dung AI sinh ra phải có thể được giảng viên kiểm duyệt và chỉnh sửa trước khi công bố cho học viên. Học viên chỉ thấy nội dung đã được giảng viên chấp thuận.

2.4.3 BR-03: Xử lý và sinh nội dung AI

Hệ thống phải tự động phân tích cấu trúc tài liệu, chia thành các đơn vị học tập có nghĩa, và sinh ra nội dung tóm tắt cùng câu hỏi kiểm tra phù hợp với nội dung gốc. Nội dung sinh ra phải ở ngôn ngữ tương ứng với tài liệu đầu vào.

2.4.4 BR-04: Trải nghiệm học tập và đánh giá

Học viên phải có thể học theo đơn vị nhỏ và tự đánh giá kiến thức sau mỗi đơn vị. Hệ thống phải lưu lại tiến độ và kết quả của học viên để phục vụ việc theo dõi và cải thiện.

2.5 Yêu cầu tính năng

Yêu cầu tính năng (Functional Requirements — FR) cụ thể hóa BR thành các chức năng hệ thống có thể đo lường.

2.5.1 FR-01: Đăng nhập / Đăng xuất / Đăng ký tài khoản

- Người dùng đăng ký bằng email + mật khẩu; xác nhận qua email.
- Đăng nhập nhận JWT access token (1 giờ) + refresh token (7 ngày).
- Mật khẩu hash bằng `bcrypt`; không lưu plaintext.
- Đăng xuất vô hiệu hóa refresh token phía server.

2.5.2 FR-02: Quản lý tài liệu

- Upload file PDF/DOCX/PPTX $\leq 50\text{MB}$; kiểm tra MIME type.
- Lưu file vào MinIO; tạo bản ghi `Document` với trạng thái `QUEUED`.
- Hiển thị danh sách tài liệu của giảng viên với trạng thái xử lý theo thời gian thực.
- Xóa tài liệu: xóa file MinIO + cascade xóa chunk, card, quiz liên quan.

2.5.3 FR-03: Xử lý tài liệu tự động (AI Pipeline)

- Sau khi upload, hệ thống tự động enqueue job và AI Pipeline worker xử lý bất đồng bộ.
- Trạng thái cập nhật theo thời gian thực qua server streaming: `QUEUED` $\rightarrow$ `PARSING` $\rightarrow$ `CHUNKING` $\rightarrow$ `GENERATING` $\rightarrow$ `DONE` (hoặc `ERROR`).
- Retry tối đa 3 lần nếu LLM API lỗi; ghi log chi tiết mỗi lần retry.
- Pipeline có thể xử lý đồng thời tối đa 10 tài liệu.

2.5.4 FR-04: Review và publish khóa học

- Giảng viên xem, sửa (inline edit) và xóa từng Card và câu Quiz.
- Thêm câu hỏi thử công vào bất kỳ section nào.
- Sắp xếp lại thứ tự Card trong section bằng drag-and-drop.
- Publish khóa học: điều kiện ≥ 1 section có `is_reviewed=true`.
- Unpublish: ẩn khóa học khỏi danh sách học viên (soft delete).

2.5.5 FR-05: Trải nghiệm học tập

- Học viên xem Card theo thứ tự; hệ thống lưu tiến độ sau mỗi Card.
- Làm Quiz: hiển thị ngẫu nhiên từ pool câu hỏi; phản hồi tức thì.
- Tìm kiếm ngữ nghĩa: vector similarity top-K.
- Xem lịch sử học: tiến độ theo khóa học, điểm Quiz theo thời gian.

2.6 Yêu cầu phi tính năng

Table 9: Yêu cầu phi tính năng (NFR).

ID	Tên	Tiêu chí cụ thể
NFR-01	Hiệu năng	Xử lý tài liệu 20 trang ≤ 3 phút; API read ≤ 500 ms ở P95; hỗ trợ 50 concurrent users xem Card/Quiz đồng thời.
NFR-02	Bảo mật	JWT trên mọi API; HTTPS bắt buộc; API key LLM qua biến môi trường; rate limiting 100 req/phút/IP; OWASP Top 10 check.
NFR-03	Độ tin cậy	Uptime $\geq 95\%$ staging; pipeline idempotent (retry an toàn); không mất dữ liệu khi lỗi giữa chừng.
NFR-04	Khả năng bảo trì	Kiến trúc DDD; unit test coverage $\geq 70\%$; tài liệu API đầy đủ (auto-generated từ <code>.proto</code>).
NFR-05	Khả năng mở rộng	Thêm parser mới chỉ cần tạo adapter; thay LLM provider không sửa use case; Docker Compose $\rightarrow$ Kubernetes khi cần.
NFR-06	Khả dụng (Usability)	Không cần đào tạo; giao diện responsive; thông báo lỗi rõ và có hướng dẫn khắc phục; SUS Score $\geq 68/100$.

2.7 Tổng kết Chương 2

Chương 2 đã trình bày toàn diện bức tranh hiện tại của bài toán: từ kết quả khảo sát thực tế với giảng viên và học viên, phân tích so sánh bốn giải pháp tương tự, đến đặc tả chi tiết user stories và các yêu cầu hệ thống (BR, FR, NFR). Các phân tích này là căn cứ để thiết kế hệ thống trong các chương tiếp theo.

3 Cơ sở lý thuyết

3.1 Lý thuyết công nghệ sử dụng

3.1.1 Mô hình Ngôn ngữ Lớn (Large Language Model)

Định nghĩa: Mô hình Ngôn ngữ Lớn (LLM) là các mạng nơ-ron sâu với hàng tỷ tham số, được huấn luyện trên tập văn bản khổng lồ để học phân phối xác suất của ngôn ngữ tự nhiên. Kiến trúc nền tảng là **Transformer** [2] với cơ chế *self-attention* cho phép nắm bắt phụ thuộc xa trong chuỗi văn bản.

Cơ chế hoạt động: Khi nhận một đầu vào (prompt), LLM sinh ra đầu ra bằng cách dự đoán token tiếp theo lặp đi lặp lại (autoregressive decoding). Tham số **temperature** điều chỉnh tính ngẫu nhiên: giá trị thấp (< 0.3) cho đầu ra nhất quán, giá trị cao (> 0.7) cho đầu ra sáng tạo hơn.

Ứng dụng trong đề tài: LLM được sử dụng cho hai tác vụ:

- **Tóm tắt (Summarization):** Từ một chunk văn bản, sinh ra danh sách 3–5 ý chính ngắn gọn.
- **Sinh câu hỏi (Question Generation):** Từ cùng chunk, sinh ra 5–10 câu hỏi trắc nghiệm 4 đáp án kèm giải thích.

3.1.2 Prompt Engineering

Định nghĩa: Prompt Engineering là kỹ thuật thiết kế đầu vào cho LLM để dẫn hướng đầu ra về chất lượng, định dạng và nội dung mong muốn, mà không cần thay đổi tham số mô hình (zero-shot / few-shot).

Các kỹ thuật áp dụng trong đề tài:

Table 10: Các kỹ thuật Prompt Engineering và ứng dụng trong đề tài.

Kỹ thuật	Mô tả	Ứng dụng
Role Prompting	Gán vai trò chuyên gia cho LLM để định hướng phong cách.	“Bạn là giảng viên đại học soạn đề kiểm tra...”
Structured Output	Yêu cầu LLM trả về JSON theo schema cố định.	<code>response_format = json_object;</code> schema kiểm tra bằng Pydantic.
Constraint Prompting	Đặt ràng buộc rõ ràng về số lượng, độ dài, ngôn ngữ.	“Tạo đúng {n} câu hỏi; mỗi câu ≤ 30 từ; ngôn ngữ: tiếng Việt.”
Few-shot Examples	Cung cấp ví dụ mẫu để định hướng format.	Đính kèm 1 ví dụ câu hỏi mẫu trong prompt.

3.1.3 Text Chunking

Định nghĩa: Text Chunking là quá trình chia văn bản dài thành các đoạn nhỏ hơn (chunk) có tính ngữ nghĩa độc lập, phục vụ cho xử lý LLM và lưu trữ vector.

Vì sao cần chunking? LLM có giới hạn *context window* (ví dụ GPT-4o-mini: 128K token). Tuy nhiên, đưa toàn bộ tài liệu vào một lần gọi API vừa tốn kém vừa làm giảm chất lượng đầu ra do LLM khó tập trung vào từng phần. Chunking giải quyết vấn đề này bằng cách xử lý từng đoạn nhỏ.

Bài toán thiết kế: Chunk lý tưởng cần thỏa mãn ba điều kiện: (1) tính ngữ nghĩa (một ý hoàn chỉnh), (2) kích thước phù hợp (500–1500 token), (3) metadata đầy đủ (thuộc section nào, trang bao nhiêu).

3.1.4 Vector Embedding

Định nghĩa: Vector Embedding là ánh xạ $f : \text{text} \rightarrow \mathbb{R}^d$ sao cho các văn bản có nghĩa tương đồng có véc-tơ gần nhau theo độ đo cosine:

$$\text{sim}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{|\vec{u}| |\vec{v}|}$$

Ứng dụng: Mỗi chunk được chuyển thành véc-tơ $\vec{e} \in \mathbb{R}^{768}$ và lưu vào Qdrant. Khi học viên tìm kiếm, query cũng được chuyển thành véc-tơ và Qdrant tìm top-K chunk có similarity cao nhất — đây là nền tảng của tìm kiếm ngữ nghĩa và RAG.

3.1.5 Domain-Driven Design (DDD)

Định nghĩa: DDD [1] là phương pháp thiết kế phần mềm tổ chức code xung quanh mô hình nghiệp vụ. Nguyên tắc cốt lõi: logic nghiệp vụ phải độc lập với framework và hạ tầng kỹ thuật.

Áp dụng trong đề tài — bốn lớp:

1. **Domain:** Entity (Document, Chunk), Port interface khai báo contract (LLMPort, ParserPort, VectorDBPort).
2. **Application:** Use case (ProcessDocumentUseCase, SearchChunksUseCase) — chỉ phụ thuộc Port.
3. **Infrastructure:** Adapter cài đặt Port (OpenAIAdapter, PDFParser, QdrantAdapter, MinIOAdapter).
4. **Delivery:** gRPC Servicer nhận request, gọi use case.

3.2 Cân nhắc lựa chọn công nghệ

3.2.1 Lựa chọn LLM API: GPT-4o-mini vs Gemini Flash

Table 11: So sánh GPT-4o-mini và Gemini Flash.

Tiêu chí	GPT-4o-mini	Gemini 1.5 Flash
Chất lượng tiếng Việt	Tốt	Tốt (hỗ trợ tốt đa ngôn ngữ)
Chi phí (input/1M token)	\$0.15	\$0.075
Tốc độ	Nhanh (~30 token/s)	Rất nhanh (~60 token/s)
JSON mode	Có (json_object)	Có (application/json)
Độ ổn định API	Rất cao	Cao

Quyết định: Sử dụng **GPT-4o-mini** làm mặc định do hệ sinh thái OpenAI ổn định hơn và tài liệu phong phú hơn. Kiến trúc adapter cho phép chuyển sang Gemini Flash dễ dàng nếu cần tối ưu chi phí.

3.2.2 Lựa chọn chiến lược Chunking

Table 12: So sánh các chiến lược Chunking cho tài liệu học thuật.

Chiến lược	Ưu điểm	Nhược điểm
Fixed-size	Đơn giản; kích thước nhất quán.	Cắt đứt câu/ý; mất ngữ cảnh.
Sentence-based	Bảo toàn câu hoàn chỉnh.	Chunk nhỏ; thiếu ngữ cảnh đoạn.
Heading-based	Phù hợp tài liệu có cấu trúc; mỗi chunk = 1 chủ đề.	Phụ thuộc chất lượng cấu trúc tài liệu gốc.
Semantic	Chính xác nhất.	Chi phí tính toán cao.

Quyết định: **Heading-based** là chiến lược chính. Section vượt ngưỡng sẽ được chia tiếp bằng sliding window với overlap 100 token để bảo toàn ngữ cảnh.

3.2.3 Lựa chọn Vector Database: Qdrant vs Weaviate vs Pinecone

Table 13: So sánh các Vector Database.

Tiêu chí	Qdrant	Weaviate	Pinecone
Tự host (on-premise)	Có	Có	Không (cloud-only)
Hiệu năng ANN	Rất cao (HNSW)	Cao	Cao
Payload filtering	Có	Có	Có (giới hạn)
Python SDK	Đơn giản	Phức tạp hơn	Đơn giản
Chi phí	Miễn phí (OSS)	Miễn phí (OSS)	Có phí (theo vector)

Quyết định: Qdrant — mã nguồn mở, tự host, hiệu năng cao, Python SDK thân thiện.

3.2.4 Lựa chọn Embedding Model: bge-m3 vs text-embedding-3-small

Table 14: So sánh mô hình Embedding.

Tiêu chí	BAAI/bge-m3	text-embedding-3-small
Chiều véc-tơ	768	1536
Hỗ trợ tiếng Việt	Rất tốt (đa ngôn ngữ)	Tốt
Chi phí	Miễn phí (local)	\$0.02/1M token
Tốc độ inference	Nhanh (CPU đủ dùng)	Rất nhanh (API call)

Quyết định: BAAI/bge-m3 chạy local — không tốn chi phí API, hỗ trợ tiếng Việt tốt, phù hợp môi trường staging.

3.3 Tổng kết Chương 3

Chương 3 đã trình bày nền tảng lý thuyết cho các công nghệ cốt lõi của hệ thống: LLM và Prompt Engineering, Text Chunking, Vector Embedding và kiến trúc DDD. Đặc biệt, phần cân nhắc lựa chọn công nghệ đã lý giải cụ thể lý do chọn GPT-4o-mini, Heading-based Chunking, Qdrant và bge-m3 so với các lựa chọn thay thế. Những quyết định này sẽ được hiện thực hóa trong Chương 4 và 5.

4 Thiết kế hệ thống

4.1 Thiết kế kiến trúc hệ thống

4.1.1 Tổng quan kiến trúc

Hệ thống Micro-learning được thiết kế theo kiến trúc **Microservices kết hợp Domain-Driven Design (DDD)**, chia thành các service độc lập. Các tác vụ đồng bộ sử dụng gRPC, trong khi các tác vụ nền kéo dài (AI Pipeline) được điều phối qua cơ chế **job queue + worker**. Toàn bộ hệ thống được thiết kế hoàn chỉnh ngay từ đầu dự án — bao gồm cả các thành phần sẽ được hiện thực ở giai đoạn sau — nhằm đảm bảo tính nhất quán kiến trúc và tránh việc phải tái cấu trúc lớn khi mở rộng.

Figure 5: Kiến trúc tổng quan hệ thống Micro-learning.

Hệ thống gồm bốn nhóm thành phần lớn:

- **Client Layer:** React SPA giao tiếp với backend qua gRPC-Web.
- **API Gateway:** Envoy Proxy chuyển đổi gRPC-Web → gRPC, xử lý TLS và rate limiting.
- **Service Layer:** Bốn service nghiệp vụ độc lập (API Service, AI Pipeline Service, Video Service, Chat Service), giao tiếp nội bộ qua gRPC cho tác vụ đồng bộ và qua job queue cho tác vụ nền.
- **Infrastructure Layer:** PostgreSQL, Qdrant, MinIO, Redis — lưu trữ dữ liệu quan hệ, vector, file, cache và hàng đợi nhẹ cho pipeline.

4.1.2 Kiến trúc phân tầng (Layered Architecture)

Hệ thống được tổ chức thành **bốn tầng** (layer), mỗi tầng có trách nhiệm rõ ràng và chỉ giao tiếp với tầng liền kề:

Figure 6: Kiến trúc phân tầng của hệ thống.

Table 15: Mô tả bốn tầng kiến trúc của hệ thống.

Tầng	Thành phần	Trách nhiệm
Client Layer	React SPA (TypeScript + Vite)	Giao diện web cho Giảng viên, Học viên, Admin. Giao tiếp với Backend qua gRPC-Web (qua Envoy proxy).
API Gateway	Envoy Proxy	Chuyển đổi gRPC-Web → gRPC; load balancing; TLS termination; rate limiting 100 req/phút/IP.
Service Layer	API Service, AI Pipeline Service, Video Service, Chat Service	Xử lý nghiệp vụ cốt lõi. Mỗi service có domain riêng, giao tiếp nội bộ qua gRPC và job queue tùy loại tác vụ.
Infrastructure Layer	PostgreSQL, MinIO, Redis, Qdrant,	Lưu trữ dữ liệu quan hệ, vector, file, semantic cache và job queue cho AI Pipeline.

4.1.3 Chi tiết các service

(1) **API Service** — Service trung tâm, đóng vai trò gateway nghiệp vụ. Nhận request từ client, xác thực JWT, kiểm tra phân quyền, sau đó điều phối đến các service khác hoặc truy cập trực tiếp database.

Chức năng chính:

- Xác thực và phân quyền (Auth).
- CRUD tài liệu, khóa học, người dùng.
- Enqueue job xử lý tài liệu/video vào Redis-backed queue.
- Phục vụ dữ liệu cho frontend (danh sách Card, Quiz, tiến độ học).
- Server-streaming endpoint để cập nhật trạng thái pipeline real-time.

(2) **AI Pipeline Service** — Service xử lý bất đồng bộ, điều phối toàn bộ pipeline sinh nội dung. Service này hoạt động như một tập worker tiêu thụ job từ hàng đợi, sau đó thực hiện Parse → Chunk → Embed → Generate đối với tài liệu text. Khi nhận video/audio, worker gọi thêm Video Service để transcribe và clip trước khi đưa transcript vào pipeline text.

(3) **Video Service** — Service chuyên biệt xử lý media:

- Nhận file video/audio từ MinIO, trích xuất audio track.
- Gọi Whisper API để tạo transcript có word-level timestamp.

- Dùng FFmpeg cắt video gốc thành clip ngắn (1–5 phút) theo timestamp của từng ý chính.
- Lưu clip vào MinIO, metadata vào bảng `media_clips`.

(4) **Chat Service** — Service RAG Chatbot hỗ trợ học viên hỏi đáp về nội dung bài học:

- Nhận câu hỏi từ học viên.
- Kiểm tra semantic cache (Redis) trước khi gọi LLM.
- Truy vấn Qdrant để tìm top-K chunk liên quan (Retrieval).
- Xây dựng RAG prompt, gọi LLM sinh câu trả lời (Generation).
- Trả lời kèm trích dẫn nguồn (chunk ID, heading, trang).
- Lưu lịch sử hội thoại để duy trì ngữ cảnh trong session.

4.1.4 Bài toán thiết kế và giải pháp

Trong quá trình thiết kế, nhóm xác định năm bài toán kỹ thuật chính cần giải quyết:

Table 16: Năm bài toán thiết kế và giải pháp tương ứng.

#	Bài toán	Nguyên nhân	Giải pháp
1	Xử lý bất đồng bộ	Pipeline AI kéo dài vài phút; HTTP request sẽ timeout.	Job Queue + Worker + server-streaming status.
2	Thay thế LLM provider	OpenAI có thể thay đổi giá/điều khoản; cần linh hoạt.	Port/Adapter pattern (Hexagonal Architecture).
3	Kiểm thử không gọi API thật	Gọi LLM API thật tốn tiền và phụ thuộc mạng.	Mock Adapter implement cùng Port interface.
4	Chatbot bịa đặt	LLM có thể sinh thông tin sai (hallucination).	RAG — chỉ trả lời dựa trên context từ Qdrant.
5	Chi phí API cao	Nhiều câu hỏi trùng lặp; token lãng phí.	Semantic caching + prompt token optimization.

4.1.4.1 Giải pháp chi tiết Bài toán 1 — Job Queue kết hợp State Machine.

Khi giảng viên upload tài liệu, API Service không gọi trực tiếp AI Pipeline bằng RPC dài hạn mà **đưa một job vào hàng đợi Redis**. AI Pipeline worker tiêu thụ job theo thứ tự, cập nhật trạng thái vào database sau mỗi bước và frontend theo dõi tiến trình

qua server streaming. Bên trong mỗi job, pipeline vẫn được mô hình hóa như một **state machine** với các trạng thái chuyển tiếp rõ ràng. Nếu bất kỳ bước nào thất bại, hệ thống chuyển sang **ERROR** và ghi nhận bước cuối cùng thành công để hỗ trợ retry từ đúng điểm đó.

Figure 7: State Machine của AI Pipeline.

Table 17: Các trạng thái trong State Machine của AI Pipeline.

Trạng thái	Mô tả	Chuyển tiếp
QUEUED	File đã lưu vào MinIO; job đã được đẩy vào queue.	→ TRANSCRIBING (video) hoặc PARSING (text) khi worker nhận job
TRANSCRIBING	Whisper đang chuyển audio/video → text.	→ PARSING hoặc ERROR
PARSING	Trích xuất text + heading từ tài liệu.	→ CHUNKING hoặc ERROR
CHUNKING	Chia text thành chunk theo heading.	→ EMBEDDING hoặc ERROR
EMBEDDING	Tạo vector embedding cho mỗi chunk.	→ GENERATING hoặc ERROR
GENERATING	Gọi LLM sinh Summary + Quiz.	→ CLIPPING (video) hoặc DONE
CLIPPING	FFmpeg cắt video thành clip ngắn (chỉ với input video).	→ DONE hoặc ERROR
DONE	Pipeline hoàn tất.	(terminal state)
ERROR	Có lỗi xảy ra; ghi <code>error_msg</code> + <code>last_success_state</code> .	Retry → quay lại trạng thái sau <code>last_success_state</code>

Cơ chế **retry**: Khi pipeline gặp lỗi tạm thời (LLM API timeout, rate limit), worker tự động retry tối đa 3 lần với exponential backoff (1s → 2s → 4s). Nếu vẫn thất bại, job được đánh dấu **ERROR**; API Service hiển thị nút “Retry” để đưa lại job vào queue từ bước cuối cùng thành công.

4.1.4.2 Giải pháp chi tiết Bài toán 2 & 3 — Hexagonal Architecture (Port/Adapter).

Áp dụng kiến trúc Port/Adapter để tách biệt nghiệp vụ khỏi hạ tầng kỹ thuật. Mỗi dependency bên ngoài (LLM, database, storage) được trừu tượng hóa thành một **Port** (interface). Khi cần thay thế provider, chỉ cần tạo Adapter mới mà không sửa bất kỳ use case nào.

Figure 8: Kiến trúc Hexagonal — Port/Adapter cho AI Pipeline.

Table 18: Danh sách Port và Adapter trong hệ thống.

Port (Interface)	Production Adapter	Test Adapter
LLMPort	OpenAIAdapter, GeminiAdapter	MockLLMAdapter (trả JSON cố định)
ParserPort	PDFParser, DocxParser, PptxParser	MockParser
EmbeddingPort	BGEm3Adapter (local)	MockEmbeddingAdapter (vector ngẫu nhiên)
VectorDBPort	QdrantAdapter	InMemoryVectorDB
StoragePort	MinIOAdapter	LocalFSAdapter
STTPort	WhisperAdapter, GoogleSTTAdapter	MockSTTAdapter
VideoPort	FFmpegAdapter	MockVideoAdapter

4.1.4.3 Giải pháp chi tiết Bài toán 4 — RAG (Retrieval-Augmented Generation) cho Chatbot Tutor. RAG là kỹ thuật kết hợp truy vấn thông tin (Retrieval) với sinh văn bản (Generation) để giảm hallucination. Chatbot chỉ trả lời dựa trên nội dung bài học đã được index trong Qdrant.

Figure 9: Kiến trúc RAG Pipeline cho Chatbot Tutor.

Luồng xử lý RAG:

1. Học viên gửi câu hỏi (ví dụ: “Giải thích thuật toán Dijkstra”).
2. Câu hỏi được chuyển thành vector embedding (bge-m3).
3. Truy vấn Qdrant: tìm top-5 chunk có cosine similarity cao nhất, **lọc theo course_id** để chỉ tìm trong nội dung khóa học đang học.
4. Xây dựng prompt: *System prompt* (vai trò giảng viên, không được bịa đặt) + *Context* (5 chunk) + *User question*.
5. Gọi LLM sinh câu trả lời; kèm trích dẫn chunk ID và heading.
6. Nếu không tìm được chunk liên quan (similarity < 0.5), trả lời: “Nội dung này chưa có trong bài học, vui lòng hỏi giảng viên.”

4.1.4.4 Giải pháp chi tiết Bài toán 5 — Tối ưu chi phí API. Ba chiến lược tối ưu được áp dụng:

1. **Semantic Caching:** Trước khi gọi LLM, kiểm tra xem câu hỏi có tương đồng (≥ 0.95 cosine) với câu hỏi đã trả lời trước đó không. Nếu có, trả về kết quả cache (Redis) thay vì gọi API mới.

2. **Prompt Compression:** Loại bỏ whitespace thừa, rút gọn context chunk (chỉ lấy phần liên quan nhất thay vì toàn bộ chunk).
3. **Model Tiering:** Dùng mô hình nhỏ hơn (GPT-4o-mini) cho tác vụ đơn giản (tóm tắt); chỉ dùng mô hình lớn (GPT-4o) cho tác vụ phức tạp (sinh câu hỏi đánh giá bậc cao Bloom).

4.1.5 Thiết kế luồng dữ liệu (Data Flow)

Figure 10: Luồng dữ liệu Text Processing: từ Upload đến Card + Quiz.

4.1.5.1 Luồng dữ liệu: Text Processing Pipeline.

1. **Upload:** Giảng viên upload file PDF/DOCX/PPTX → API Service lưu file vào MinIO, tạo record `documents` với trạng thái `QUEUED`.
2. **Enqueue:** API Service đẩy job vào Redis queue và trả ngay `document_id` cho frontend.
3. **Consume:** AI Pipeline worker nhận job từ queue, lấy file từ MinIO → Parser trích xuất raw text + heading structure → lưu parsed content tạm thời.
4. **Chunk:** Chunker chia parsed text thành các chunk theo heading; section dài được sliding window → lưu vào bảng `chunks`.
5. **Embed:** Mỗi chunk được chuyển thành vector 768 chiều (bge-m3) → upsert vào Qdrant collection `document_chunks` với payload metadata.
6. **Generate:** Từng batch chunk được gửi đến LLM (OpenAI API) → sinh Summary (3–5 ý chính) + Quiz (5–10 câu MCQ) → lưu vào bảng `cards` và `quiz_questions`.
7. **Notify:** Cập nhật trạng thái `DONE` → `WatchStatus` stream thông báo giảng viên.

Figure 11: Luồng dữ liệu: từ Video Upload đến Clip + Card + Quiz.

4.1.5.2 Luồng dữ liệu: Video/Audio Processing Pipeline.

1. **Upload Video:** Giảng viên upload file MP4/MP3/WAV → lưu vào MinIO bucket `media`.
2. **Transcribe:** Video Service gọi Whisper API → nhận transcript có timestamp cho từng câu/đoạn.

3. **Segment:** Dựa trên transcript, AI phân tích và xác định các ý chính kèm timestamp bắt đầu–kết thúc.
4. **Clip:** FFmpeg cắt video gốc thành các clip ngắn (1–5 phút) theo timestamp → lưu clip vào MinIO.
5. **Process:** Transcript của mỗi clip được đưa qua Text Pipeline (Chunk → Embed → Generate) như với tài liệu text.
6. **Output:** Mỗi section gồm: video clip + Card tóm tắt + Quiz.

Figure 12: Luồng dữ liệu RAG Chatbot.

4.1.5.3 Luồng dữ liệu: RAG Chatbot.

1. Học viên gửi câu hỏi qua Chat UI.
2. Chat Service kiểm tra **semantic cache** (Redis):
 - Cache hit (≥ 0.95 similarity): trả về câu trả lời đã cache.
 - Cache miss: tiếp tục bước 3.
3. Embed câu hỏi → truy vấn Qdrant (filter `course_id`) → lấy top-5 chunk.
4. Xây dựng RAG prompt → gọi LLM → sinh câu trả lời.
5. Lưu câu hỏi + câu trả lời vào **semantic cache** và **chat history** (PostgreSQL).
6. Trả kết quả cho học viên kèm trích dẫn nguồn.

4.2 Thiết kế giao tiếp giữa các service (API Design)

4.2.1 Tổng quan giao thức gRPC

Tất cả giao tiếp giữa các service sử dụng **gRPC** với Protocol Buffers. Client (React) giao tiếp qua **gRPC-Web** thông qua Envoy proxy.

Lý do chọn gRPC thay vì REST:

- **Strongly typed:** Schema định nghĩa trong `.proto`, tự động sinh code cho cả Python và TypeScript.
- **Hiệu năng:** Binary encoding nhỏ hơn JSON $\sim 3\text{--}5\times$.
- **Streaming:** Hỗ trợ server-side streaming cho cập nhật pipeline status real-time.
- **Code generation:** Giảm sai sót do manual serialization.

4.2.2 Định nghĩa Service Protobuf

Table 19: Định nghĩa RPC cho Auth Service.

RPC Method	Request / Response	Re-	Mô tả
Register	RegisterReq UserResp	→	Đăng ký tài khoản mới (email + password + role).
Login	LoginReq TokenPair	→	Xác thực, trả access_token + refresh_token.
RefreshToken	RefreshReq TokenPair	→	Gia hạn access_token.
Logout	LogoutReq → Empty		Vô hiệu hóa refresh_token.

4.2.2.1 Auth Service.

Table 20: Định nghĩa RPC cho Document Service.

RPC Method	Request / Response	Mô tả
UploadDocument	UploadDocumentReq → DocumentResp	Upload tài liệu, tạo bản ghi QUEUED và enqueue job xử lý.
GetDocument	DocIdReq → DocumentResp	Lấy thông tin tài liệu.
ListDocuments	ListDocsReq → DocListResp	Danh sách tài liệu (phân trang).
DeleteDocument	DocIdReq → Empty	Xóa tài liệu + cascade.
WatchStatus	DocIdReq → stream StatusUpdate	Server streaming: pipeline status real-time.

4.2.2.2 Document Service.

Table 21: Định nghĩa RPC cho Content Service.

RPC Method	Request / Response	Mô tả
GetCards	DocIdReq → CardListResp	Lấy danh sách Card theo tài liệu.
UpdateCard	UpdateCardReq → CardResp	Giảng viên chỉnh sửa Card.
GetQuizzes	ChunkIdReq → QuizListResp	Lấy Quiz theo chunk.
UpdateQuiz	UpdateQuizReq → QuizResp	Chỉnh sửa câu hỏi Quiz.
PublishCourse	CourseIdReq → CourseResp	Publish khóa học.
ReorderCards	ReorderReq → Empty	Sắp xếp lại thứ tự Card.

4.2.2.3 Content Service.

Table 22: Định nghĩa RPC cho Learning Service.

RPC Method	Request / Response	Mô tả
ListCourses	ListCoursesReq → CourseListResp	Danh sách khóa học (published).
GetLessonCards	CourseIdReq → CardListResp	Lấy Card để học.
SubmitQuizAnswer	AnswerReq → AnswerResp	Nộp đáp án, nhận phản hồi.
GetProgress	ProgressReq → ProgressResp	Tiến độ học tập.
SemanticSearch	SearchReq → SearchResp	Tìm kiếm ngữ nghĩa (Qdrant).

4.2.2.4 Learning Service.

Table 23: Định nghĩa RPC cho Chat Service.

RPC Method	Request / Response	Mô tả
AskQuestion	ChatReq → stream ChatResp	Hỏi chatbot; streaming response.
GetChatHistory	SessionReq → ChatHistoryResp	Lịch sử hội thoại.
RateAnswer	RatingReq → Empty	Đánh giá chất lượng trả lời (thumbs up/down).

4.2.2.5 Chat Service.

4.3 Thiết kế cơ sở dữ liệu

4.3.1 Lược đồ ERD tổng quan (hai giai đoạn)

Figure 13: Lược đồ ERD tổng quan. Bảng in nghiêng chỉ áp dụng khi input là video/audio.

4.3.2 Các bảng nghiệp vụ cốt lõi

Table 24: Mô tả bảng `users`.

Cột	Kiểu	Mô tả
id	UUID PK	Khóa chính tự sinh (gen_random_uuid).
email	VARCHAR UNIQUE	Địa chỉ email đăng nhập.
password_hash	VARCHAR	Bcrypt hash của mật khẩu.
role	ENUM	INSTRUCTOR LEARNER ADMIN.
full_name	VARCHAR(100)	Họ tên đầy đủ.
is_active	BOOLEAN	Tài khoản chưa bị khóa.
created_at	TIMESTAMPTZ	Thời điểm tạo.

Table 25: Mô tả bảng `documents`.

Cột	Kiểu	Mô tả
id	UUID PK	Khóa chính.
owner_id	UUID FK	Tham chiếu <code>users.id</code> .
title	VARCHAR(255)	Tên tài liệu.
file_key	TEXT	Key object trong MinIO bucket.
file_type	ENUM	PDF DOCX PPTX MP4 MP3.
media_type	ENUM	TEXT VIDEO AUDIO — phân loại input.
language	ENUM	vi en.
status	ENUM	State machine: QUEUED → ... → DONE.
last_success_state	ENUM NULL	Trạng thái cuối thành công (hỗ trợ retry).
error_msg	TEXT NULL	Thông báo lỗi (nếu có).
total_tokens_used	INT DEFAULT 0	Tổng token đã dùng (tracking chi phí).
created_at	TIMESTAMPTZ	Thời điểm tạo.

Table 26: Mô tả bảng `chunks`.

Cột	Kiểu	Mô tả
<code>id</code>	UUID PK	Khóa chính; đồng thời là point ID trong Qdrant.
<code>document_id</code>	UUID FK	Tài liệu gốc.
<code>content</code>	TEXT	Nội dung text của chunk.
<code>heading</code>	TEXT	Tiêu đề section.
<code>chunk_index</code>	INT	Thứ tự trong tài liệu.
<code>token_count</code>	INT	Token ước tính (cl100k_base tokenizer).
<code>page_start</code>	INT NULL	Trang bắt đầu trong tài liệu gốc.
<code>page_end</code>	INT NULL	Trang kết thúc.
<code>timestamp_start</code>	FLOAT NULL	(chỉ với video) Giờ bắt đầu.
<code>timestamp_end</code>	FLOAT NULL	(chỉ với video) Giờ kết thúc.

Table 27: Mô tả bảng `cards` và `quiz_questions`.

Cột	Kiểu	Mô tả
<i>Bảng <code>cards</code></i>		
<code>id</code>	UUID PK	Khóa chính.
<code>chunk_id</code>	UUID FK	Chunk tương ứng.
<code>summary</code>	JSONB	["ý chính 1", "ý chính 2", ...].
<code>is_reviewed</code>	BOOLEAN	Giảng viên đã review.
<code>sort_order</code>	INT	Thứ tự hiển thị (hỗ trợ drag-and-drop).
<i>Bảng <code>quiz_questions</code></i>		
<code>id</code>	UUID PK	Khóa chính.
<code>chunk_id</code>	UUID FK	Chunk nguồn.
<code>question</code>	TEXT	Câu hỏi.
<code>options</code>	JSONB	{"A": "...", "B": "...", "C": "...", "D": "..."}.
<code>answer</code>	CHAR(1)	Đáp án đúng.
<code>explanation</code>	TEXT	Giải thích.
<code>difficulty</code>	ENUM	EASY MEDIUM HARD.
<code>bloom_level</code>	ENUM NULL	Bloom's Taxonomy level.

Table 28: Mô tả bảng `courses`.

Cột	Kiểu	Mô tả
<code>id</code>	UUID PK	Khóa chính.
<code>document_id</code>	UUID FK	Tài liệu nguồn.
<code>instructor_id</code>	UUID FK	Giảng viên sở hữu.
<code>title</code>	VARCHAR(255)	Tên khóa học.
<code>description</code>	TEXT NULL	Mô tả ngắn.
<code>is_published</code>	BOOLEAN	Đã publish cho học viên.
<code>published_at</code>	TIMESTAMPTZ NULL	Thời điểm publish.
<code>created_at</code>	TIMESTAMPTZ	Thời điểm tạo.

4.3.3 Các bảng bổ sung cho Video và Chatbot

Table 29: Mô tả bảng `media_clips`.

Cột	Kiểu	Mô tả
<code>id</code>	UUID PK	Khóa chính.
<code>chunk_id</code>	UUID FK	Chunk tương ứng (1 clip = 1 chunk).
<code>file_key</code>	TEXT	Key trong MinIO bucket <code>media</code> .
<code>duration_sec</code>	FLOAT	Thời lượng clip (giây).
<code>format</code>	ENUM	MP4 WEBM.
<code>created_at</code>	TIMESTAMPTZ	Thời điểm tạo.

Table 30: Mô tả bảng `transcripts`.

Cột	Kiểu	Mô tả
<code>id</code>	UUID PK	Khóa chính.
<code>document_id</code>	UUID FK	Video/audio gốc.
<code>full_text</code>	TEXT	Toàn bộ transcript.
<code>segments</code>	JSONB	[{"start": 0.0, "end": 5.2, "text": "..."}, ...].
<code>language</code>	ENUM	Ngôn ngữ phát hiện được.
<code>model_used</code>	VARCHAR(50)	Whisper (tiny/base/small/medium/large). model
<code>created_at</code>	TIMESTAMPTZ	Thời điểm tạo.

Table 31: Mô tả bảng chat_sessions và chat_messages.

Cột	Kiểu	Mô tả
<i>Bảng chat_sessions</i>		
id	UUID PK	Khóa chính.
learner_id	UUID FK	Học viên.
course_id	UUID FK	Khóa học đang hỏi.
created_at	TIMESTAMPTZ	Thời điểm bắt đầu.
<i>Bảng chat_messages</i>		
id	UUID PK	Khóa chính.
session_id	UUID FK	Phiên hội thoại.
role	ENUM	USER ASSISTANT.
content	TEXT	Nội dung tin nhắn.
source_chunks	UUID[] NULL	Danh sách chunk_id đã trích dẫn.
tokens_used	INT	Token tiêu thụ.
rating	SMALLINT NULL	Đánh giá: 1 (thumbs down) hoặc 5 (thumbs up).
created_at	TIMESTAMPTZ	Thời điểm gửi.

Table 32: Mô tả bảng learning_progress.

Cột	Kiểu	Mô tả
id	UUID PK	Khóa chính.
learner_id	UUID FK	Học viên.
course_id	UUID FK	Khóa học.
card_id	UUID FK	Card đã xem.
completed_at	TIMESTAMPTZ	Thời điểm hoàn thành.

Table 33: Mô tả bảng quiz_attempts.

Cột	Kiểu	Mô tả
id	UUID PK	Khóa chính.
learner_id	UUID FK	Học viên.
question_id	UUID FK	Câu hỏi.
selected	CHAR(1)	Đáp án đã chọn.
is_correct	BOOLEAN	Đúng hay sai.
answered_at	TIMESTAMPTZ	Thời điểm trả lời.

4.3.4 Mức độ chuẩn hóa và trade-off

Lược đồ được thiết kế đạt **Chuẩn thứ ba (3NF)**:

- **1NF**: Mọi cột chỉ chứa giá trị nguyên tử; không có group lặp (trừ JSONB là intentional).

- **2NF:** Mọi thuộc tính không khóa phụ thuộc đầy đủ vào khóa chính (không có partial dependency).
- **3NF:** Không có phụ thuộc bắc cầu (transitive dependency).

Trade-off có chủ đích:

1. **JSONB cho summary, options, segments:** Vi phạm 1NF theo nghĩa chặt, nhưng giữ nguyên vì: (a) cấu trúc mảng cố định, không cần truy vấn từng phần tử riêng lẻ; (b) tránh tạo bảng join không cần thiết; (c) JSONB trong PostgreSQL hỗ trợ GIN index và query hiệu quả.
2. **UUID[] cho source_chunks:** Dùng mảng thay vì bảng trung gian vì: (a) chỉ cần hiển thị, không cần join ngược; (b) số lượng chunk trích dẫn luôn ≤ 10 .
3. **Tách courses khỏi documents:** Một document có thể tạo nhiều version khóa học; cho phép instructor tạo lại pipeline mà không mất khóa học cũ.

4.3.5 Chiến lược index

Table 34: Danh sách index quan trọng.

Index	Bảng.Cột	Lý do
idx_docs_owner	documents.owner_id	Giảng viên list tài liệu của mình.
idx_chunks_doc	chunks.document_id	Lấy tất cả chunk theo tài liệu.
idx_cards_chunk	cards.chunk_id	Join Card với Chunk.
idx_quiz_chunk	quiz_questions.chunk_id	Lấy Quiz theo chunk.
idx_progress_learner	learning_progress (learner_id, course_id)	Tính tiến độ học.
idx_attempts_learner	quiz_attempts (learner_id, question_id)	Kiểm tra đã trả lời chưa.
idx_chat_session	chat_messages.session_id	Lưu lịch sử chat.

4.4 Thiết kế Sequence Diagram

4.4.1 Sequence Diagram: Upload và xử lý tài liệu

Figure 14: Sequence Diagram: Giảng viên upload tài liệu và pipeline xử lý.

Mô tả luồng:

1. Giảng viên chọn file và nhấn Upload trên giao diện.
2. React gửi file upload request đến API Service.
3. API Service lưu file vào MinIO, tạo record **documents** (status = QUEUED).
4. API Service đẩy một job vào Redis queue để bắt đầu xử lý và trả về **document_id** cho client ngay lập tức.
5. Client mở kết nối server streaming (**WatchStatus**) để nhận cập nhật trạng thái.
6. AI Pipeline worker nhận job từ queue và thực hiện tuần tự: Parse → Chunk → Embed → Generate. Sau mỗi bước, worker cập nhật **status** trong database và API Service stream trạng thái mới về client.
7. Khi hoàn tất (**DONE**), client hiển thị nút “Review”.

4.4.2 Sequence Diagram: Học viên làm Quiz

Figure 15: Sequence Diagram: Học viên làm Quiz và nhận phản hồi.

Mô tả luồng:

1. Học viên mở trang Quiz của một section.
2. Client gọi **GetQuizzes** → nhận danh sách câu hỏi (đã shuffle ngẫu nhiên).
3. Học viên chọn đáp án → client gọi **SubmitQuizAnswer**.
4. API Service kiểm tra đáp án, tạo record **quiz_attempts**, trả về **is_correct** + **explanation**.
5. Client hiển thị phản hồi tức thì (highlight xanh/đỏ + giải thích).
6. Sau khi làm hết, client gọi **GetProgress** để cập nhật tiến độ tổng thể.

4.4.3 Sequence Diagram: RAG Chatbot

Figure 16: Sequence Diagram: Học viên hỏi Chatbot RAG.

Mô tả luồng:

1. Học viên nhập câu hỏi trong Chat UI.

2. Client gọi `AskQuestion` (server streaming).
3. Chat Service kiểm tra semantic cache (Redis):
 - Cache hit: stream câu trả lời từ cache, kết thúc.
 - Cache miss: tiếp tục.
4. Chat Service embed câu hỏi (bge-m3) → truy vấn Qdrant (filter `course_id`, top-5).
5. Nếu max similarity < 0.5: trả lời “Nội dung không có trong bài học”.
6. Nếu có kết quả: xây dựng RAG prompt (system + context chunks + chat history gần nhất + câu hỏi).
7. Gọi LLM API (streaming) → stream từng token về client.
8. Lưu message vào `chat_messages`, cache vào Redis.

4.4.4 Sequence Diagram: Video Processing

Figure 17: Sequence Diagram: Xử lý Video.

Mô tả luồng:

1. Giảng viên upload video → lưu MinIO, status = `QUEUED`.
2. API Service đẩy job video vào queue; AI Pipeline worker nhận job và phát hiện `media_type = VIDEO` → gọi Video Service.
3. Video Service tải video từ MinIO → trích xuất audio track.
4. Gọi Whisper API: audio → transcript (word-level timestamp).
5. Lưu transcript vào bảng `transcripts`.
6. AI Pipeline nhận transcript → gọi LLM để phân tích ý chính và xác định timestamp ranh giới giữa các ý.
7. Video Service dùng FFmpeg cắt video gốc thành clip theo timestamp → lưu clip vào MinIO, metadata vào `media_clips`.
8. Transcript mỗi clip được đưa vào Text Pipeline (Chunk → Embed → Generate) như tài liệu text thông thường.

4.5 Thiết kế bảo mật

4.5.1 Xác thực và phân quyền

Figure 18: Luồng xác thực JWT.

Hệ thống sử dụng **JWT (JSON Web Token)** với cơ chế dual-token:

- **Access Token** (thời hạn 1 giờ): Chứa `user_id`, `role`, `exp`. Gửi kèm mỗi request trong header `Authorization: Bearer <token>`.
- **Refresh Token** (thời hạn 7 ngày): Dùng để lấy access token mới khi hết hạn. Lưu ở server (database) để có thể revoke khi logout.

Phân quyền (Authorization): Áp dụng **Role-Based Access Control (RBAC)** với ba vai trò:

Table 35: Ma trận phân quyền theo vai trò.

Hành động	Instructor	Learner	Admin
Upload tài liệu	✓		✓
Review/Edit nội dung	✓		
Publish khóa học	✓		
Xem Card/Quiz	✓	✓	✓
Làm Quiz		✓	
Tìm kiếm ngữ nghĩa	✓	✓	✓
Hỏi Chatbot		✓	
Quản lý người dùng			✓
Xem thống kê chi phí			✓

4.5.2 Bảo mật dữ liệu và API

- **HTTPS/TLS:** Bắt buộc cho mọi kết nối client-server. TLS termination tại Envoy proxy.
- **Mật khẩu:** Hash bằng `bcrypt` (cost factor = 12); không bao giờ lưu plaintext.
- **API Key bảo mật:** LLM API key (OpenAI, Google) lưu trong biến môi trường, không hardcode.
- **Rate Limiting:** 100 request/phút/IP tại Envoy proxy; 10 request/phút cho endpoint Upload (tránh spam pipeline).
- **Input Validation:** Kiểm tra MIME type file upload; giới hạn kích thước 50MB; sanitize mọi input text.

- **CORS:** Chỉ cho phép origin cụ thể (domain frontend).
- **SQL Injection Prevention:** Sử dụng parameterized queries (SQLAlchemy ORM); không bao giờ string concatenation.

4.6 Thiết kế triển khai (Deployment Architecture)

4.6.1 Docker Compose topology

Figure 19: Docker Compose topology — tất cả service chạy trong container.

Table 36: Danh sách Docker containers.

Container	Image	Port	Ghi chú
frontend	node:20-alpine	3000	React dev server (Vite).
envoy	envoyproxy/envoy	8080	gRPC-Web → gRPC proxy.
api	python:3.11-slim	50051	API Service (gRPC).
ai-pipeline	python:3.11-slim	50052	AI Pipeline worker consume job queue.
postgres	postgres:16	5432	Database chính.
qdrant	qdrant/qdrant	6333	Vector database.
minio	minio/minio	9000	Object storage.
redis	redis:7-alpine	6379	Semantic cache cho chatbot và job queue cho pipeline.

4.6.2 Network và Volume

- **Network:** Tất cả container cùng Docker bridge network `ml-network`. Service giao tiếp nội bộ qua container name (DNS resolution); API và worker cùng truy cập Redis để enqueue/ consume job.
- **Volumes:** Persistent volume cho PostgreSQL (`./data/postgres`), Qdrant (`./data/qdrant`), MinIO (`./data/minio`). Đảm bảo dữ liệu không mất khi restart.
- **Environment:** Mọi secret (API key, DB password) qua `.env` file, không commit vào git.

4.6.3 Định hướng mở rộng sang Kubernetes (sau GD2)

Kiến trúc Docker Compose hiện tại được thiết kế sẵn sàng chuyển sang Kubernetes khi cần scale:

- Mỗi service là một container độc lập → dễ tạo Deployment/Pod.

- gRPC giữa các service → Kubernetes Service discovery tương thích.
- Stateful data (Postgres, Qdrant, MinIO) dùng PersistentVolumeClaim.
- AI Pipeline có thể scale horizontal (nhiều worker xử lý đồng thời nhiều document).

4.7 Thiết kế giao diện

4.7.1 Bộ nhận diện thương hiệu

Table 37: Bảng màu sắc và typography của hệ thống.

Thành phần	Giá trị	Ý nghĩa
Màu chính (Primary)	#2563EB (Blue-600)	Hành động chính, nút CTA.
Màu phụ (Secondary)	#10B981 (Emerald-500)	Thành công, hoàn thành.
Màu cảnh báo (Warning)	#F59E0B (Amber-500)	Cần review, đang xử lý.
Màu lỗi (Error)	#EF4444 (Red-500)	Lỗi, thất bại.
Nền (Background)	#F9FAFB (Gray-50)	Nền trang.
Font chính	Inter (sans-serif)	Văn bản giao diện.
Font code	JetBrains Mono	Hiển thị code/JSON.

4.7.2 Sơ đồ trang (Sitemap)

Figure 20: Sơ đồ trang (Sitemap) của toàn bộ hệ thống (hai giai đoạn).

Giảng viên:

- `/login, /register`
- `/dashboard` — Tổng quan
- `/documents` — Danh sách tài liệu
- `/documents/upload` — Upload mới (text, video, audio)
- `/documents/:id/status` — Theo dõi tiến trình
- `/documents/:id/review` — Review và chỉnh sửa
- `/courses/:id/publish` — Publish khóa học

- `/courses/:id/analytics` — Thống kê học viên

Học viên:

- `/courses` — Danh sách khóa học
- `/courses/:id` — Chi tiết khóa học
- `/courses/:id/learn` — Xem Card (có thể kèm video clip)
- `/courses/:id/quiz` — Làm Quiz
- `/courses/:id/chat` — Chatbot RAG
- `/profile` — Lịch sử học tập
- `/search` — Tìm kiếm ngữ nghĩa

4.7.3 Wireframe các màn hình chính

Figure 21: Wireframe Dashboard Giảng viên — hiển thị tổng quan tài liệu và trạng thái pipeline.

4.7.3.1 Wireframe 1: Dashboard Giảng viên. Layout gồm: (1) Sidebar navigation; (2) Thống kê tổng quan (số tài liệu, số khóa học published, số học viên); (3) Bảng danh sách tài liệu với cột trạng thái hiển thị badge màu theo trạng thái job/pipeline.

Figure 22: Wireframe Review — layout 2 cột: bên trái là text gốc, bên phải là nội dung AI sinh.

4.7.3.2 Wireframe 2: Màn hình Review nội dung AI. Layout 2 cột: (1) Cột trái hiển thị chunk text gốc (read-only, highlight heading); (2) Cột phải hiển thị Summary Card (inline editable) và danh sách Quiz (editable). Có nút “Mark as Reviewed” cho từng section.

Figure 23: Wireframe Card view — một Card hiển thị tại một thời điểm, navigation bằng phím mũi tên.

4.7.3.3 Wireframe 3: Màn hình học Card. Card view: hiển thị 1 card tại 1 thời điểm; progress bar phía trên; nút Previous/Next; badge section heading; nội dung tóm tắt dạng bullet points. Với khóa học từ video, card có thể kèm embedded video clip.

Figure 24: Wireframe Chatbot — panel bên phải trong màn hình học.

4.7.3.4 Wireframe 4: Chatbot UI. Chatbot hiển thị dạng slide-over panel bên phải màn hình học. Gồm: lịch sử tin nhắn (bubble style), input box phía dưới, mỗi câu trả lời có trích dẫn nguồn (clickable link đến Card tương ứng), nút thumbs up/down để đánh giá.

4.8 Thiết kế Use Case Diagram

Figure 25: Use Case Diagram tổng quan (hai giai đoạn).

4.8.1 Nhóm use case Xác thực

Figure 26: Use Case Diagram — Xác thực người dùng.

Gồm: Đăng ký, Đăng nhập, Đăng xuất, Refresh Token. Actor: tất cả người dùng chưa đăng nhập.

4.8.2 Nhóm use case Quản lý tài liệu và nội dung

Figure 27: Use Case Diagram — Quản lý tài liệu và nội dung AI.

Gồm: Upload tài liệu (text, video, audio), Theo dõi pipeline, Review nội dung, Chỉnh sửa Card/Quiz, Publish/Unpublish khóa học. Actor: Instructor.

4.8.3 Nhóm use case Học tập

Figure 28: Use Case Diagram — Học tập và kiểm tra.

Gồm: Duyệt khóa học, Xem Card, Xem video clip (nếu có), Làm Quiz, Tìm kiếm ngữ nghĩa, Hỏi Chatbot, Xem tiến độ. Actor: Learner.

4.8.4 Nhóm use case Quản trị

Figure 29: Use Case Diagram — Quản trị hệ thống.

Gồm: Quản lý người dùng (CRUD, khóa/mở tài khoản), Xem thống kê chi phí API, Xem log hệ thống. Actor: Admin.

4.9 Thiết kế Activity Diagram

4.9.1 Quy trình upload và xử lý tài liệu

Figure 30: Activity Diagram: Upload và xử lý tài liệu.

Luồng chính: Chọn file → Validate (MIME, size) → Upload MinIO → Enqueue Job → [Fork] Frontend theo dõi status || Worker xử lý tuần tự → [Join khi DONE] → Hiển thị nút Review.

Luồng ngoại lệ: File không hợp lệ → thông báo lỗi. Pipeline lỗi → hiển thị error + nút Retry.

4.9.2 Quy trình AI Pipeline (Job Queue + State Machine)

Figure 31: Activity Diagram: AI Pipeline chi tiết (Text và Video).

Worker lấy job từ queue, sau đó decision node đầu tiên kiểm tra `media_type`:

- TEXT → Parse → Chunk → Embed → Generate → Done.
- VIDEO/AUDIO → Transcribe (Whisper) → Parse Transcript → Chunk → Embed → Generate → Clip Video (FFmpeg) → Done.

Mỗi bước có guard condition: thành công → tiếp tục; thất bại → retry (max 3) → nếu vẫn thất bại → ERROR state.

4.9.3 Quy trình review và publish

Figure 32: Activity Diagram: Review và publish khóa học.

Luồng: Mở trang Review → Xem từng section (chunk gốc + AI output) → [Decision] Cần sửa? → Sửa inline / Xóa / Thêm câu hỏi → Mark Reviewed → Kiểm tra ≥ 1 section reviewed? → [Yes] Enable Publish button → Publish.

4.9.4 Quy trình học viên học bài và làm Quiz

Figure 33: Activity Diagram: Học viên xem Card, xem Video (nếu có) và làm Quiz.

Luồng: Chọn khóa học → Xem Card → [Card có video clip? → Xem video clip] → Next Card → [Hết Card] → Làm Quiz → Chọn đáp án → Nhận phản hồi → [Hết câu] → Xem kết quả tổng → Cập nhật progress.

4.9.5 Quy trình Chatbot RAG

Figure 34: Activity Diagram: Học viên hỏi Chatbot RAG.

Luồng: Nhập câu hỏi → Check semantic cache → [Hit] Trả lời từ cache → End. [Miss] → Embed câu hỏi → Query Qdrant → [Có kết quả ≥ 0.5 ?] → [Yes] Build RAG prompt → Call LLM (streaming) → Hiển thị + Cache. [No] → Trả lời “Nội dung không có trong bài học”.

4.10 Tổng kết Chương 4

Chương 4 đã trình bày **toàn bộ bản thiết kế kỹ thuật** của hệ thống cho **cả hai giai đoạn**, bao gồm:

1. **Kiến trúc hệ thống:** Kiến trúc phân tầng 4 layer, chi tiết từng service, giải pháp cho 5 bài toán thiết kế (job queue, Hexagonal Architecture, RAG, semantic caching), luồng dữ liệu chi tiết cho cả text pipeline và video/audio pipeline.
2. **Thiết kế API:** Định nghĩa gRPC service cho Auth, Document, Content, Learning và Chat với request/response type cụ thể.
3. **Cơ sở dữ liệu:** 12 bảng đầy đủ, chuẩn hóa 3NF với trade-off JSONB có chủ đích, chiến lược index.
4. **Sequence Diagram:** 4 luồng chính (Upload Pipeline, Quiz, RAG Chatbot, Video Processing).
5. **Bảo mật:** JWT dual-token, RBAC 3 vai trò, HTTPS, rate limiting, input validation.
6. **Triển khai:** Docker Compose 8 container, network và volume, định hướng Kubernetes.



7. **Giao diện:** Branding, Sitemap, 4 wireframe chính.

8. **UML:** Use case diagram (4 nhóm), Activity diagram (5 quy trình).

Thiết kế này là nền tảng để hiện thực hệ thống trong Chương 5 (GD1) và tiếp tục mở rộng trong Giai đoạn 2.

5 Hiện thực hệ thống

5.1 Nền tảng sử dụng

5.1.1 Python 3.11 và gRPC

Python 3.11 là ngôn ngữ chính cho AI Pipeline. Lý do chọn Python: hệ sinh thái ML/AI phong phú nhất (`sentence-transformers`, `pdfplumber`, `openai`); cộng đồng lớn; `asyncio` native hỗ trợ I/O-bound workload hiệu quả.

gRPC + Protobuf: Giao thức giao tiếp giữa các service. Ưu điểm so với REST/JSON: binary encoding nhanh hơn $\sim 3\text{--}5\times$, schema strongly typed qua `.proto`, hỗ trợ streaming response.

5.1.2 MinIO Object Storage

MinIO là hệ thống object storage mã nguồn mở tương thích S3 API. Trong đề tài, MinIO lưu trữ file tài liệu gốc (PDF, DOCX, PPTX). Bucket chính: `documents`. Mỗi file có key: `{owner_id}/{doc_id}/original.{ext}`.

5.1.3 Qdrant Vector Database

Qdrant chạy trong Docker container, persistent volume tại `./data/qdrant`. Collection `document_chunks`: vector 768 chiều, metric cosine, payload gồm `document_id`, `chunk_id`, `heading`, `language`.

5.1.4 Redis Cache và Job Queue

Redis được sử dụng cho hai mục đích. Thứ nhất, Redis lưu semantic cache cho Chatbot để giảm số lần gọi LLM với các câu hỏi tương tự. Thứ hai, Redis đóng vai trò hàng đợi nhẹ cho AI Pipeline: API Service enqueue job sau khi upload tài liệu, còn AI Pipeline worker consume job và cập nhật trạng thái xử lý sau mỗi bước.

5.1.5 React + TypeScript — Frontend

Frontend được xây dựng bằng **React 18 + TypeScript + Vite**. Thư viện UI: **Tailwind CSS + shadcn/ui** (headless components). Giao tiếp với Backend qua **gRPC-Web client** thông qua Envoy proxy.

5.1.6 Docker và Docker Compose

Hệ thống được đóng gói bằng Docker Compose với các service chính gồm API, AI Pipeline worker, Frontend, Qdrant, MinIO, Postgres, Redis và Envoy proxy. Cách tổ chức này giúp

triển khai đồng nhất giữa môi trường phát triển và môi trường demo, đồng thời đơn giản hóa việc cấu hình mạng nội bộ giữa các service.

5.2 Hiện thực AI Pipeline

5.2.1 Module Document Parsing

Ba parser được triển khai thông qua cùng một interface trừu tượng để bảo đảm tính nhất quán trong luồng xử lý đầu vào. Mỗi parser nhận file gốc và trả về một cấu trúc tài liệu đã chuẩn hóa, bao gồm văn bản, heading và thông tin phân đoạn. Với PDF, hệ thống sử dụng `pdfplumber` để trích xuất nội dung và heuristic dựa trên font size hoặc định dạng chữ nhằm phát hiện heading.

5.2.2 Module Text Chunking

Sau khi parsing, tài liệu được chia thành các chunk theo cấu trúc heading. Những section ngắn được giữ nguyên, còn section dài được cắt bằng cơ chế sliding window có overlap để tránh mất ngữ cảnh ở ranh giới giữa các đoạn. Cách làm này giúp cân bằng giữa độ đầy đủ ngữ nghĩa và giới hạn token đầu vào của mô hình sinh nội dung.

5.2.3 Module sinh nội dung AI

Tầng sinh nội dung AI được hiện thực thông qua adapter kết nối tới LLM. Từ mỗi chunk đầu vào, hệ thống sinh ra hai loại đầu ra chính: tập ý tóm tắt ngắn gọn và bộ câu hỏi quiz trắc nghiệm. Các prompt được chuẩn hóa theo định dạng JSON để giảm sai khác cấu trúc đầu ra và đơn giản hóa bước kiểm tra dữ liệu ở backend.

5.2.4 Use Case ProcessDocument

Use case trung tâm của hệ thống điều phối toàn bộ pipeline xử lý tài liệu. Sau khi người dùng tải file lên, API Service lưu file vào MinIO, tạo bản ghi với trạng thái `QUEUED` và đẩy một job vào Redis queue. AI Pipeline worker sau đó consume job, lần lượt thực hiện các bước parse, chunking, sinh embedding, sinh nội dung AI và lưu kết quả vào cơ sở dữ liệu. Trạng thái xử lý của tài liệu được cập nhật sau từng giai đoạn để frontend có thể hiển thị tiến trình theo thời gian thực. Ở bước sinh nội dung, các chunk được xử lý theo batch nhằm giảm nguy cơ vượt rate limit và tận dụng khả năng xử lý song song.

5.3 Hiện thực Web Application

5.3.1 Giao diện Giảng viên

Figure 35: Màn hình Dashboard Giảng viên.

Figure 36: Màn hình Upload tài liệu.

Figure 37: Màn hình theo dõi tiến trình xử lý AI (4 bước).

Figure 38: Màn hình Review nội dung AI (layout 2 cột).

5.3.2 Giao diện Học viên

Figure 39: Màn hình danh sách khóa học của Học viên.

Figure 40: Màn hình xem bài học dạng Card.

Figure 41: Màn hình làm Quiz (hiển thị phản hồi tức thì sau khi chọn đáp án).

5.4 Tổng kết Chương 5

Chương 5 đã trình bày chi tiết việc hiện thực hệ thống: lý giải lựa chọn từng nền tảng công nghệ, mã nguồn cốt lõi của AI Pipeline (Parser, Chunker, LLM Adapter, Use Case) và giao diện Web Application cho cả hai nhóm người dùng. Toàn bộ mã nguồn được tổ chức theo kiến trúc DDD, đảm bảo tính modular và dễ kiểm thử — cơ sở để tiến hành kiểm thử toàn diện trong Chương 6.

6 Kiểm thử hệ thống

6.1 Kế hoạch kiểm thử

6.1.1 Mục tiêu

- Xác minh tất cả các chức năng đã đặc tả (FR-01 đến FR-05) hoạt động đúng theo tiêu chí chấp nhận.
- Đánh giá hệ thống đáp ứng các yêu cầu phi tính năng (NFR-01 đến NFR-06).
- Đánh giá chất lượng nội dung AI sinh ra (Summary và Quiz).
- Xác định các điểm yếu cần cải thiện trước Giai đoạn 2.

6.1.2 Phương pháp

Table 38: Các phương pháp kiểm thử được áp dụng.

Phương pháp	Công cụ	Phạm vi
Unit Testing	pytest, unittest.mock	Từng module (Chunker, Parser, Adapter).
Integration Testing	pytest + Docker Compose	Luồng end-to-end thực tế.
Black-box Testing	Thủ công theo test case	FR-01 đến FR-05.
Load Testing	locust	NFR-01 (hiệu năng).
AI Quality Evaluation	Khảo sát giảng viên (Likert 1–5)	Chất lượng Summary và Quiz.
Usability Testing	SUS Questionnaire (10 sinh viên)	NFR-06 (khả dụng).

6.1.3 Phạm vi

Trong phạm vi: Toàn bộ FR và NFR đã đặc tả ở Chương 2.

Ngoài phạm vi: Kiểm thử bảo mật chuyên sâu (penetration testing); kiểm thử tương thích trình duyệt (chỉ test Chrome/Firefox).

6.2 Kiểm thử tính năng (Functional Testing)

6.2.1 Cấu hình môi trường kiểm thử

Table 39: Cấu hình môi trường kiểm thử tính năng.

Thành phần	Cấu hình
Môi trường	Docker Compose (local), giống production
Dữ liệu test	10 tài liệu mẫu: 3 PDF, 3 DOCX, 2 PPTX, 1 PDF lỗi, 1 PDF vượt kích thước
Tài khoản test	2 Giảng viên, 5 Học viên, 1 Admin
LLM	GPT-4o-mini (API key thật, giới hạn \$5 budget)

6.2.2 Kiểm thử Hộp đen (Black-box Testing)

Table 40: Test cases cho FR-01: Xác thực người dùng.

ID	Mô tả	Input	Expected	Result
TC-01-01	Đăng nhập với email và mật khẩu hợp lệ	email: test@bk.edu.vn, password: Test@123	JWT token trả về; redirect dashboard	
TC-01-02	Đăng nhập với mật khẩu sai	password: Wrong- Pass	HTTP 401; thông báo “Sai email hoặc mật khẩu”	
TC-01-03	Đăng nhập với tài khoản bị khóa	Tài khoản is_active=false	HTTP 403; thông báo “Tài khoản bị khóa”	
TC-01-04	Truy cập API không có JWT	Không có Autho- rization header	HTTP 401; “Unauthorized”	
TC-01-05	JWT hết hạn (access token)	JWT expired 2 giờ trước	HTTP 401; “To- ken expired”	

Table 41: Test cases cho FR-02/03: Upload và xử lý tài liệu.

ID	Mô tả	Input	Expected	Result
TC-02-01	Upload PDF hợp lệ (10 trang)	chapter1.pdf, 2MB	Document tạo với status QUEUED; job được enqueue và pipeline khởi động	
TC-02-02	Upload file sai định dạng (.xlsx)	data.xlsx	HTTP 400; “Định dạng không hỗ trợ”	
TC-02-03	Upload file vượt kích thước (60MB)	large.pdf, 60MB	HTTP 413; “File tối đa 50MB”	
TC-02-04	Pipeline hoàn tất: kiểm tra số chunk	PDF 10 trang, 5 section	≥ 5 chunk được tạo	
TC-02-05	Pipeline hoàn tất: kiểm tra Card	Mỗi chunk phải có đúng 1 Card	Số Card = số chunk	
TC-02-06	Pipeline hoàn tất: kiểm tra Quiz	Mỗi chunk phải có ≥ 5 câu Quiz	Số Quiz $\geq 5 \times$ số chunk	
TC-02-07	LLM API lỗi network: kiểm tra retry	Giả lập LLM time-out	Retry đúng 3 lần; status = ERROR sau lần 3	

Table 42: Test cases cho FR-04/05: Review, publish và học tập.

ID	Mô tả	Input	Expected	Result
TC-04-01	Chỉnh sửa nội dung Card	Sửa ý chính thứ 2	Thay đổi lưu ngay; nội dung mới xuất hiện	
TC-04-02	Publish khi chưa review bất kỳ section	Nhấn Publish	Nút Publish disabled; tooltip giải thích	
TC-04-03	Publish sau khi review 1 section	Review section 1; nhấn Publish	Khóa học status = PUBLISHED; hiện trong list học viên	



ID	Mô tả	Input	Expected	Result
TC-05-01	Học viên làm Quiz: chọn đúng	Chọn đáp án đúng	Highlight xanh; hiển thị giải thích; điểm +1	
TC-05-02	Học viên làm Quiz: chọn sai	Chọn đáp án sai	Highlight đỏ; hiện đáp án đúng và giải thích	
TC-05-03	Tiến độ được lưu khi thoát giữa chừng	Thoát ở Card thứ 3/10	Quay lại tiếp tục từ Card thứ 3	

6.3 Kiểm thử phi tính năng

6.3.1 Cấu hình môi trường kiểm thử

Table 43: Cấu hình môi trường kiểm thử phi tính năng.

Thành phần	Cấu hình
Công cụ load test	locust 2.x
Máy chủ	VM: 4 CPU, 8GB RAM (giả lập staging)
Dữ liệu	100 khóa học, mỗi khóa 10 section, 50 Card, 50 câu Quiz
Mạng	Localhost (latency <1ms)

6.3.2 Kiểm thử hiệu năng (Benchmark & Load Test)

Table 44: Kết quả kiểm thử hiệu năng API (50 concurrent users, locust).

Endpoint	RPS	Avg (ms)	P50 (ms)	P95 (ms)	Error %
GET /cards (xem Card)					
GET /quiz (lấy Quiz)					
POST /progress (lưu tiến độ)					
GET /search (vector search)					

Table 45: Kết quả kiểm thử thời gian xử lý pipeline (tài liệu thực tế).

Tài liệu	Trang	Chunks	Parse (s)	Embed (s)	LLM (s)
PDF 10 trang	10				
PDF 25 trang	25				
DOCX 30 trang	30				
PPTX 20 slides	20				
Tiêu chí	Tổng ≤ 3 phút cho 20 trang (NFR-01)				

6.3.3 Kiểm thử chất lượng nội dung AI

Phương pháp: ... giảng viên đánh giá độc lập 20 Card Summary và 20 câu Quiz sinh ra từ tài liệu thực tế, theo thang Likert 5 điểm.

Table 46: Kết quả đánh giá chất lượng nội dung AI (N=... giảng viên).

Tiêu chí đánh giá	Summary	Quiz	Nhận xét
Độ chính xác nội dung ($\bar{x}/5$)			
Độ rõ ràng, dễ hiểu ($\bar{x}/5$)			
Phù hợp độ khó ($\bar{x}/5$)			Chỉ áp dụng Quiz
Tỷ lệ không cần chỉnh sửa (%)			
Tiêu chí đặt ra	$\geq 70\%$	$\geq 65\%$	

Usability Testing (SUS): Thực hiện với ... sinh viên; yêu cầu hoàn thành 3 task: (1) đăng ký tài khoản, (2) học một section, (3) làm Quiz. Điền bảng SUS 10 câu sau khi hoàn thành.

Table 47: Kết quả SUS và task completion.

Chỉ số	Kết quả	Nhận xét
SUS Score	.../100	≥ 68 : Acceptable
Task completion rate	...%	Task 1/2/3: .../.../...%
Avg time trên task Quiz	...s	
Điểm hài lòng chung (1–5)	...	

6.4 Tổng kết Chương 6

Chương 6 đã trình bày toàn diện kế hoạch và kết quả kiểm thử hệ thống theo hai nhóm: kiểm thử tính năng (black-box test cases cho FR-01 đến FR-05) và kiểm thử phi tính năng (load test, đánh giá chất lượng AI, SUS usability). Kết quả kiểm thử xác nhận ... (nhóm điền tổng kết sau khi có số liệu). Các điểm cần cải thiện sẽ được ưu tiên trong kế hoạch Giai đoạn 2.

7 Kết luận

7.1 Kết quả đạt được

Sau 15 tuần thực hiện, Giai đoạn 1 đã hoàn thành đầy đủ các mục tiêu đề ra:

Table 48: Tổng kết kết quả Giai đoạn 1 theo tiêu chí đề ra.

Tiêu chí (từ Bảng 3)	Ngưỡng	Đạt được	Trạng thái
Thời gian xử lý 20 trang	≤ 3 phút	... phút	Đạt
Tỷ lệ Summary đạt chất lượng	$\geq 70\%$	... %	
Tỷ lệ Quiz đạt chất lượng	$\geq 65\%$	... %	
API P95 (xem Card/Quiz)	$\leq 500\text{ms}$	... ms	
SUS Score giao diện	$\geq 68/100$	... /100	
Coverage unit test	$\geq 70\%$	... %	

Tóm tắt đóng góp kỹ thuật:

- Xây dựng pipeline AI hoàn chỉnh 5 bước theo kiến trúc DDD với khả năng thay thế LLM provider mà không sửa use case.
- Module Heading-based Chunker xử lý đúng cấu trúc tài liệu học thuật tiếng Việt.
- Tích hợp vector search với `bge-m3` + Qdrant phục vụ tìm kiếm ngữ nghĩa đa ngôn ngữ.
- Giao diện Human-in-the-Loop đảm bảo giảng viên kiểm duyệt trước khi nội dung đến học viên.

7.2 Điểm cần cải thiện

1. **Parsing tài liệu phức tạp:** PDF có bảng biểu nhiều cột, công thức toán học (LaTeX), hình ảnh nhúng chữ chưa được trích xuất chính xác. Cần tích hợp OCR (Tesseract) và MathPix cho Giai đoạn 2.
2. **Chi phí LLM:** Mỗi tài liệu 25 trang tốn khoảng \$... — cần caching và tối ưu prompt để giảm chi phí.
3. **Giao diện chưa hoàn thiện:** Một số UX flow còn thô (drag- and-drop sort chưa mượt); chưa responsive hoàn toàn trên mobile.
4. **Thiếu cá nhân hóa:** Học viên chưa thể hỏi đáp trực tiếp với nội dung bài học (cần RAG Chatbot ở Giai đoạn 2).

7.3 Định hướng phát triển

Giai đoạn 2 (Tuần 16–30) sẽ phát triển theo hai hướng:

1. Mở rộng đầu vào — Video Processing (Tuần 16–20):

- Tích hợp OpenAI Whisper để chuyển bài giảng video/audio thành transcript.
- Dùng FFmpeg cắt video thành clip ngắn (≤ 90 giây) dựa trên timestamp ý chính trong transcript.
- Mỗi Card sẽ có thể đính kèm clip video tương ứng.

2. Cá nhân hóa — Chatbot RAG (Tuần 21–25):

- Chatbot “Hỏi đáp bài học” cho học viên.
- RAG pipeline: khi học viên đặt câu hỏi, hệ thống tìm top-K chunk liên quan trong Qdrant, gửi cùng câu hỏi vào LLM để sinh câu trả lời chính xác, có căn cứ — tránh hallucination.
- Hiện thị nguồn trích dẫn (Card/section tương ứng) dưới mỗi câu trả lời.

3. Tối ưu hiệu năng và chi phí (Tuần 26–28):

- Redis caching cho embedding và LLM response trùng lặp.
- Tối ưu độ dài prompt; batch embedding để giảm số lần gọi API.
- Load testing với 200 concurrent users; tối ưu query PostgreSQL.



Tài liệu tham khảo

- [1] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2004.
- [2] Ashish Vaswani, Noam Shazeer, et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Phụ lục

Đường dẫn quan trọng

Tài nguyên	Đường dẫn
Repository AI Pipeline	https://github.com/...
Repository Frontend	https://github.com/...
Báo cáo (LaTeX source)	https://github.com/...
Video Demo	https://...
Figma Mockup	https://figma.com/...

Bảng đường dẫn quan trọng.

Use Case Descriptions chi tiết

UC-01: Upload tài liệu

Đặc tả UC-01.

Use Case ID	UC-01
Tên	Upload tài liệu
Tác nhân chính	Giảng viên
Tác nhân phụ	AI Pipeline
Tóm tắt	Giảng viên tải lên file tài liệu để pipeline AI xử lý thành nội dung Micro-learning.
Kích hoạt	Giảng viên chọn “Upload tài liệu mới” từ Dashboard.
Tiền điều kiện	Giảng viên đã đăng nhập với role INSTRUCTOR; JWT còn hạn.
Hậu điều kiện thành công	File lưu trong MinIO; Document bản ghi tạo với status QUEUED; job AI Pipeline được enqueue.
Hậu điều kiện thất bại	File không lưu; không có bản ghi nào được tạo; thông báo lỗi hiển thị.

Luồng chính	- Giảng viên chọn “Upload tài liệu mới”. - Hệ thống hiển thị form: tên tài liệu, mô tả, ngôn ngữ (vi/en), file đính kèm. - Giảng viên điền thông tin, kéo thả hoặc chọn file. - Frontend kiểm tra phía client: định dạng (.pdf/.docx/.pptx) và kích thước ($\leq 50\text{MB}$). - Giảng viên nhấn “Bắt đầu xử lý”. - Frontend gọi gRPC UploadDocument để gửi request upload. - Backend lưu file vào MinIO; tạo bản ghi Document. - Backend đưa job xử lý vào queue cho AI Pipeline worker. - Frontend redirect sang màn hình theo dõi tiến trình.
Luồng thay thế	- Sai định dạng (phía client): thông báo file; “Chỉ nhận PDF, DOCX, PPTX”. - Quá kích thước: “File tối đa 50MB, file hiện tại: X MB”. - MinIO lỗi: rollback; HTTP 503; “Lỗi lưu trữ, thử lại sau”. - Mất mạng khi upload: Frontend hiển thị nút “Thử lại”; giữ nguyên form.

UC-02: Xử lý tài liệu tự động (AI Pipeline)

Đặc tả UC-02.

Use Case ID	UC-02
Tên	Xử lý tài liệu tự động
Tác nhân	AI Pipeline (system actor; kích hoạt sau UC-01)
Tiền điều kiện	File tài liệu tồn tại trong MinIO; Document.status = QUEUED.

Hậu điều kiện thành công	Đủ Card và Quiz cho mọi chunk; Document.status = DONE.
Luồng chính	1. AI Pipeline worker nhận job từ queue; tải file từ MinIO. 2. Cập nhật status → PARSING; chọn parser theo file type. 3. Parser trích xuất text và cấu trúc heading. 4. Cập nhật status → CHUNKING; chia text thành chunks. 5. Lưu chunks vào PostgreSQL. 6. Cập nhật status → GENERATING. 7. Với mỗi batch (5 chunks): song song hóa embedding + LLM call. 8. Lưu vector vào Qdrant; lưu Card và Quiz vào PostgreSQL. 9. Cập nhật status → DONE; gửi cập nhật qua WatchStatus stream cho Giảng viên.
Luồng thay thế	3a. File hỏng/không đọc được: status → ERROR; log lỗi chi tiết. 7a. LLM API timeout: exponential backoff retry (1s, 2s, 4s); sau 3 lần → ERROR. 7b. LLM trả về JSON không hợp lệ: retry với prompt điều chỉnh (tối đa 2 lần).



UC-03 đến UC-08 (Tóm tắt)

Danh mục tóm tắt tất cả Use Case.

ID	Tên	Tác nhân	Kết quả chính
UC-03	Review & chỉnh sửa	Giảng viên	Card/Quiz được cập nhật; section marked reviewed.
UC-04	Publish khóa học	Giảng viên	Course.status = PUBLISHED.
UC-05	Xem Card	Học viên	Tiến độ lưu; LearningProgress cập nhật.
UC-06	Làm Quiz	Học viên	Điểm tính và lưu; QuizSubmission tạo.
UC-07	Tìm kiếm ngữ nghĩa	Học viên	Top-K Card liên quan trả về.
UC-08	Quản lý tài khoản	Admin	User CRUD; role management.